Text to speech pricing

Help and feedback

REFERENCE

Support and help options

What is text to speech?

08/07/2025

In this overview, you learn about the benefits and capabilities of the text to speech feature of the Speech service, which is part of Azure AI services.

Text to speech enables your applications, tools, or devices to convert text into human like synthesized speech. The text to speech capability is also known as speech synthesis. Use human like standard voices out of the box, or create a custom voice that's unique to your product or brand. For a full list of supported voices, languages, and locales, see Language and voice support for the Speech service.

Core features

Text to speech includes the following features:

Expand table

Feature	Summary	Demo
Standard voice (called *Neural* on the pricing page)	Highly natural out-of-the-box voices. Create an Azure subscription and Speech resource, and then use the Speech SDK or visit the Speech Studio portal and select standard voices to get started. Check the pricing details.	Check the Voice Gallery and determine the right voice for your business needs.
Custom voice	Easy-to-use self-service for creating a natural brand voice, with limited access for responsible use. Create an Azure subscription and Azure AI Foundry resource and then apply to use custom voice. After you're granted access, go to the professional voice fine-tuning documentation to get started. Check the pricing details.	Check the voice samples.

More about neural text to speech features

Text to speech uses deep neural networks to make the voices of computers nearly indistinguishable from the recordings of people. With the clear articulation of words, neural text to speech significantly reduces listening fatigue when users interact with AI systems.

The patterns of stress and intonation in spoken language are called *prosody*. Traditional text to speech systems break down prosody into separate linguistic analysis and acoustic prediction steps governed by independent models. That can result in muffled, buzzy voice synthesis.

Here's more information about neural text to speech features in the Speech service, and how they overcome the limits of traditional text to speech systems:

- **Real-time speech synthesis**: Use the Speech SDK or REST API to convert text to speech by using standard voices or custom voices.

- **Asynchronous synthesis of long audio**: Use the batch synthesis API to asynchronously synthesize text to speech files longer than 10 minutes (for example, audio books or lectures). Unlike synthesis performed via the Speech SDK or Speech to text REST API, responses aren't returned in real-time. The expectation is that requests are sent asynchronously, responses are polled for, and synthesized audio is downloaded when the service makes it available.

- **Standard voices**: Azure AI Speech uses deep neural networks to overcome the limits of traditional speech synthesis regarding stress and intonation in spoken language. Prosody prediction and voice synthesis happen simultaneously, which results in more fluid and natural-sounding outputs. Each standard voice model is available at 24 kHz and high-fidelity 48 kHz. You can use neural voices to:
  - Make interactions with chatbots and voice assistants more natural and engaging.
  - Convert digital texts such as e-books into audiobooks.
  - Enhance in-car navigation systems.

  For a full list of standard Azure AI Speech neural voices, see Language and voice support for the Speech service.

- **Improve text to speech output with SSML**: Speech Synthesis Markup Language (SSML) is an XML-based markup language used to customize text to speech outputs. With SSML, you can adjust pitch, add pauses, improve pronunciation, change speaking rate, adjust volume, and attribute multiple voices to a single document.

  You can use SSML to define your own lexicons or switch to different speaking styles. With the multilingual voices, you can also adjust the speaking languages via SSML. To improve the voice output for your scenario, see Improve synthesis with Speech Synthesis Markup Language and Speech synthesis with the Audio Content Creation tool.

- **Visemes**: Visemes are the key poses in observed speech, including the position of the lips, jaw, and tongue in producing a particular phoneme. Visemes have a strong correlation with voices and phonemes.

  By using viseme events in Speech SDK, you can generate facial animation data. This data can be used to animate faces in lip-reading communication, education, entertainment, and customer service. Viseme is currently supported only for the `en-US` (US English) neural voices.

> **Note**
>
> In addition to Azure AI Speech neural (non HD) voices, you can also use Azure AI Speech high definition (HD) voices and Azure OpenAI neural (HD and non HD) voices. The HD voices provide a higher quality for more versatile scenarios.
>
> Some voices don't support all Speech Synthesis Markup Language (SSML) tags. This includes neural text to speech HD voices, personal voices, and embedded voices.

- For Azure AI Speech high definition (HD) voices, check the SSML support here.
- For personal voice, you can find the SSML support here.
- For embedded voices, check the SSML support here.

Get started

To get started with text to speech, see the quickstart. Text to speech is available via the Speech SDK, the REST API, and the Speech CLI.

> **Tip**
>
> To convert text to speech with a no-code approach, try the **Audio Content Creation** tool in **Speech Studio**.

Sample code

Sample code for text to speech is available on GitHub. These samples cover text to speech conversion in most popular programming languages:

- Text to speech samples (SDK)
- Text to speech samples (REST)

Custom voice

In addition to standard voices, you can create custom voices that are unique to your product or brand. Custom voice is an umbrella term that includes professional voice fine-tuning and personal voice. All it takes to get started is a handful of audio files and the associated transcriptions. For more information, see the professional voice fine-tuning documentation.

Pricing note

Billable characters

When you use the text to speech feature, you're billed for each character that's converted to speech, including punctuation. Although the SSML document itself isn't billable, optional elements that are used to adjust how the text is converted to speech, like phonemes and pitch, are counted as billable characters. Here's a list of what's billable:

- Text passed to the text to speech feature in the SSML body of the request
- All markup within the text field of the request body in the SSML format, except for `<speak>` and `<voice>` tags
- Letters, punctuation, spaces, tabs, markup, and all white-space characters
- Every code point defined in Unicode

For detailed information, see Speech service pricing.

> **Important**
>
> Each Chinese character is counted as two characters for billing, including kanji used in Japanese, hanja used in Korean, or hanzi used in other languages.

Model training and hosting time for custom voice

Custom voice training and hosting are both calculated by hour and billed per second. For the billing unit price, see Speech service pricing.

Professional voice fine-tuning time is measured by "compute hour" (a unit to measure machine running time). Typically, when training a voice model, two computing tasks are running in parallel. So, the calculated compute hours are longer than the actual training time. For professional voice fine-tuning, it usually takes 20 to 40 compute hours to train a single-style voice, and around 90 compute hours to train a multi-style voice. The professional voice fine-tuning time is billed with a cap of 96 compute hours. So in the case that a voice model is trained in 98 compute hours, you'll only be charged with 96 compute hours.

Custom voice endpoint hosting is measured by the actual time (hour). The hosting time (hours) for each endpoint is calculated at 00:00 UTC every day for the previous 24 hours. For example, if the endpoint has been active for 24 hours on day one, it's billed for 24 hours at 00:00 UTC the second day. If the endpoint is newly created or suspended during the day, it's billed for its accumulated running time until 00:00 UTC the second day. If the endpoint isn't currently hosted, it isn't billed. In addition to the daily calculation at 00:00 UTC each day, the billing is

also triggered immediately when an endpoint is deleted or suspended. For example, for an endpoint created at 08:00 UTC on December 1, the hosting hour will be calculated to 16 hours at 00:00 UTC on December 2 and 24 hours at 00:00 UTC on December 3. If the user suspends hosting the endpoint at 16:30 UTC on December 3, the duration (16.5 hours) from 00:00 to 16:30 UTC on December 3 will be calculated for billing.

Personal voice

When you use the personal voice feature, you're billed for both profile storage and synthesis.

- **Profile storage**: After a personal voice profile is created, it will be billed until it's removed from the system. The billing unit is per voice per day. If voice storage lasts for less than 24 hours, it's still billed as one full day.
- **Synthesis**: Billed per character. For details on billable characters, see the above billable characters.

Text to speech avatar

When you use the text-to-speech avatar feature, charges are billed per second based on the length of video output. However, for the real-time avatar, charges are billed per second based on the time when the avatar is active, regardless of whether it's speaking or remaining silent. To optimize costs for real-time avatar usage, refer to the "Use Local Video for Idle" tips provided in the avatar chat sample code.

Custom text to speech avatar training is time is measured by "compute hour" (machine running time) and billed per second. Training duration varies depending on how much data you use. It normally takes 20-40 compute hours on average to train a custom avatar. The avatar training time is billed with a cap of 96 compute hours. So in the case that an avatar model is trained in 98 compute hours, you're only charged for 96 compute hours.

Avatar hosting is billed per second per endpoint. You can suspend your endpoint to save costs. If you want to suspend your endpoint, you can delete it directly. To use it again, redeploy the endpoint.

Monitor Azure text to speech metrics

Monitoring key metrics associated with text to speech services is crucial for managing resource usage and controlling costs. This section guides you on how to find usage information in the Azure portal and provide detailed definitions of the key metrics. For more information on Azure monitor metrics, see Azure Monitor Metrics overview.

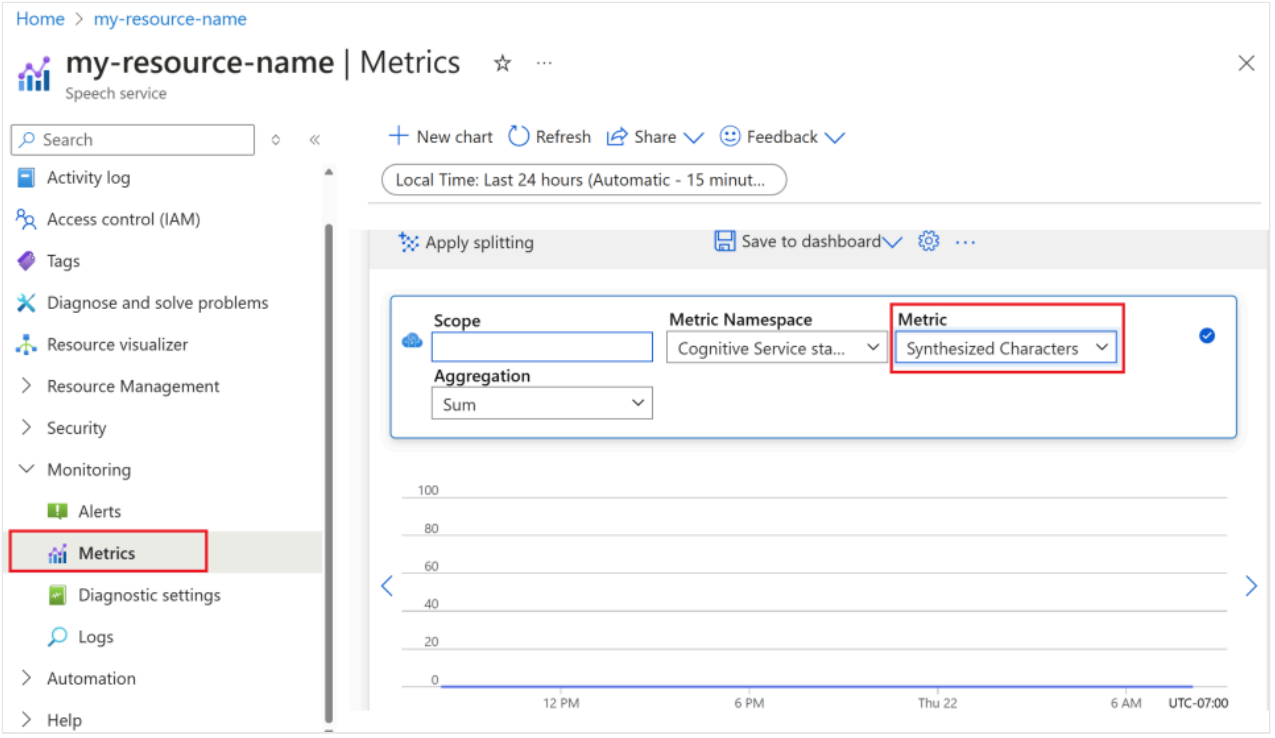
How to find usage information in the Azure portal

To effectively manage your Azure resources, it's essential to access and review usage information regularly. Here's how to find the usage information:

1. Go to the Azure portal and sign in with your Azure account.

2. Navigate to **Resources** and select your resource you wish to monitor.

3. Select **Metrics** under **Monitoring** from the left-hand menu.



4. Customize metric views.

   You can filter data by resource type, metric type, time range, and other parameters to create custom views that align with your monitoring needs. Additionally, you can save the metric view to dashboards by selecting **Save to dashboard** for easy access to frequently used metrics.

5. Set up alerts.

   To manage usage more effectively, set up alerts by navigating to the **Alerts** tab under **Monitoring** from the left-hand menu. Alerts can notify you when your usage reaches specific thresholds, helping to prevent unexpected costs.

Definition of metrics

Here's a table summarizing the key metrics for Azure text to speech.

Expand table

Metric name	Description
Synthesized Characters	Tracks the number of characters converted into speech, including standard voice and custom voice. For details on billable characters, see Billable characters.
Video Seconds Synthesized	Measures the total duration of video synthesized, including batch avatar synthesis, real-time avatar synthesis, and custom avatar synthesis.
Avatar Model Hosting Seconds	Tracks the total time in seconds that your custom avatar model is hosted.
Voice Model Hosting Hours	Tracks the total time in hours that your custom voice model is hosted.
Voice Model Training Minutes	Measures the total time in minutes for training your custom voice model.

Reference docs

- Speech SDK
- REST API: Text to speech

Responsible AI

An AI system includes not only the technology, but also the people who use it, the people who are affected by it, and the environment in which it's deployed. Read the transparency notes to learn about responsible AI use and deployment in your systems.

- Transparency note and use cases for custom voice
- Characteristics and limitations for using custom voice
- Limited access to custom voice
- Guidelines for responsible deployment of synthetic voice technology
- Disclosure for voice talent
- Disclosure design guidelines
- Disclosure design patterns
- Code of Conduct for Text to speech integrations
- Data, privacy, and security for custom voice

Next steps

- Text to speech quickstart

- Get the Speech SDK

Quickstart: Convert text to speech

07/17/2025

Reference documentation | Package (NuGet) | Additional samples on GitHub

With Azure AI Speech, you can run an application that synthesizes a human-like voice to read text. You can change the voice, enter text to be spoken, and listen to the output on your computer's speaker.

> **Tip**
>
> You can try text to speech in the Speech Studio Voice Gallery without signing up or writing any code.

> **Tip**
>
> Try out the Azure AI Speech Toolkit to easily build and run samples on Visual Studio Code.

Prerequisites

- ✓ An Azure subscription. You can create one for free.
- ✓ Create an AI Services resource for Speech in the Azure portal.
- ✓ Get the Speech resource key and endpoint. After your Speech resource is deployed, select **Go to resource** to view and manage keys.

Set up the environment

The Speech SDK is available as a NuGet package that implements .NET Standard 2.0. Install the Speech SDK later in this guide by using the console. For detailed installation instructions, see Install the Speech SDK.

Set environment variables

You need to authenticate your application to access Azure AI services. This article shows you how to use environment variables to store your credentials. You can then access the environment variables from your code to authenticate your application. For production, use a more secure way to store and access your credentials.

> **Important**
>
> We recommend Microsoft Entra ID authentication with managed identities for Azure resources to avoid storing credentials with your applications that run in the cloud.
>
> Use API keys with caution. Don't include the API key directly in your code, and never post it publicly. If using API keys, store them securely in Azure Key Vault, rotate the keys regularly, and restrict access to Azure Key Vault using role based access control and network access restrictions. For more information about using API keys securely in your apps, see API keys with Azure Key Vault.
>
> For more information about AI services security, see Authenticate requests to Azure AI services.

To set the environment variables for your Speech resource key and endpoint, open a console window, and follow the instructions for your operating system and development environment.

- To set the `SPEECH_KEY` environment variable, replace *your-key* with one of the keys for your resource.
- To set the `ENDPOINT` environment variable, replace *your-endpoint* with one of the endpoints for your resource.

Windows

Console

```
setx SPEECH_KEY your-key
setx ENDPOINT your-endpoint
```

> **Note**
>
> If you only need to access the environment variables in the current console, you can set the environment variable with `set` instead of `setx`.

After you add the environment variables, you might need to restart any programs that need to read the environment variables, including the console window. For example, if you're using Visual Studio as your editor, restart Visual Studio before you run the example.

Create the application

Follow these steps to create a console application and install the Speech SDK.

1. Open a command prompt window in the folder where you want the new project. Run this command to create a console application with the .NET CLI.

   .NET CLI

   ```
   dotnet new console
   ```

   The command creates a *Program.cs* file in the project directory.

2. Install the Speech SDK in your new project with the .NET CLI.

   .NET CLI

   ```
   dotnet add package Microsoft.CognitiveServices.Speech
   ```

3. Replace the contents of *Program.cs* with the following code.

   C#

   ```
   using System;
   using System.IO;
   using System.Threading.Tasks;
   using Microsoft.CognitiveServices.Speech;
   using Microsoft.CognitiveServices.Speech.Audio;

   class Program
   {
       // This example requires environment variables named "SPEECH_KEY" and
   "END_POINT"
       static string speechKey =
   Environment.GetEnvironmentVariable("SPEECH_KEY");
       static string endpoint = Environment.GetEnvironmentVariable("END_POINT");

       static void OutputSpeechSynthesisResult(SpeechSynthesisResult
   speechSynthesisResult, string text)
       {
           switch (speechSynthesisResult.Reason)
           {
               case ResultReason.SynthesizingAudioCompleted:
                   Console.WriteLine($"Speech synthesized for text: [{text}]");
                   break;
               case ResultReason.Canceled:
                   var cancellation =
   SpeechSynthesisCancellationDetails.FromResult(speechSynthesisResult);
                   Console.WriteLine($"CANCELED: Reason={cancellation.Reason}");

                   if (cancellation.Reason == CancellationReason.Error)
                   {
   ```

```
                    Console.WriteLine($"CANCELED: ErrorCode=
{cancellation.ErrorCode}");
                    Console.WriteLine($"CANCELED: ErrorDetails=
[{cancellation.ErrorDetails}]");
                    Console.WriteLine($"CANCELED: Did you set the speech
resource key and endpoint values?");
                }
                break;
            default:
                break;
        }
    }

    async static Task Main(string[] args)
    {
        var speechConfig = SpeechConfig.FromEndpoint(speechKey, endpoint);

        // The neural multilingual voice can speak different languages based
on the input text.
        speechConfig.SpeechSynthesisVoiceName = "en-US-
AvaMultilingualNeural";

        using (var speechSynthesizer = new SpeechSynthesizer(speechConfig))
        {
            // Get text from the console and synthesize to the default
speaker.
            Console.WriteLine("Enter some text that you want to speak >");
            string text = Console.ReadLine();

            var speechSynthesisResult = await
speechSynthesizer.SpeakTextAsync(text);
            OutputSpeechSynthesisResult(speechSynthesisResult, text);
        }

        Console.WriteLine("Press any key to exit...");
        Console.ReadKey();
    }
}
```

4. To change the speech synthesis language, replace `en-US-AvaMultilingualNeural` with another supported voice.

   All neural voices are multilingual and fluent in their own language and English. For example, if the input text in English is *I'm excited to try text to speech* and you set `es-ES-ElviraNeural` as the language, the text is spoken in English with a Spanish accent. If the voice doesn't speak the language of the input text, the Speech service doesn't output synthesized audio.

5. Run your new console application to start speech synthesis to the default speaker.

   ```
   Console
   ```

```
dotnet run
```

> **Important**
>
> Make sure that you set the `SPEECH_KEY` and `END_POINT` environment variables. If you don't set these variables, the sample fails with an error message.

6. Enter some text that you want to speak. For example, type *I'm excited to try text to speech*. Select the **Enter** key to hear the synthesized speech.

```
Console

Enter some text that you want to speak >
I'm excited to try text to speech
```

Remarks

More speech synthesis options

This quickstart uses the `SpeakTextAsync` operation to synthesize a short block of text that you enter. You can also use long-form text from a file and get finer control over voice styles, prosody, and other settings.

- See how to synthesize speech and Speech Synthesis Markup Language (SSML) overview for information about speech synthesis from a file and finer control over voice styles, prosody, and other settings.
- See batch synthesis API for text to speech for information about synthesizing long-form text to speech.

OpenAI text to speech voices in Azure AI Speech

OpenAI text to speech voices are also supported. See OpenAI text to speech voices in Azure AI Speech and multilingual voices. You can replace `en-US-AvaMultilingualNeural` with a supported OpenAI voice name such as `en-US-FableMultilingualNeural`.

Clean up resources

You can use the Azure portal or Azure Command Line Interface (CLI) to remove the Speech resource you created.

Next step

Learn more about speech synthesis

How to synthesize speech from text

08/07/2025

Reference documentation | Package (NuGet) | Additional samples on GitHub

In this how-to guide, you learn common design patterns for doing text to speech synthesis.

For more information about the following areas, see What is text to speech?

- Getting responses as in-memory streams.
- Customizing output sample rate and bit rate.
- Submitting synthesis requests by using Speech Synthesis Markup Language (SSML).
- Using neural voices.
- Subscribing to events and acting on results.

Select synthesis language and voice

The text to speech feature in the Speech service supports more than 400 voices and more than 140 languages and variants. You can get the full list or try them in the Voice Gallery.

Specify the language or voice of `SpeechConfig` to match your input text and use the specified voice. The following code snippet shows how this technique works:

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");
    // Set either the `SpeechSynthesisVoiceName` or `SpeechSynthesisLanguage`.
    speechConfig.SpeechSynthesisLanguage = "en-US";
    speechConfig.SpeechSynthesisVoiceName = "en-US-AvaMultilingualNeural";
}
```

All neural voices are multilingual and fluent in their own language and English. For example, if the input text in English, is "I'm excited to try text to speech," and you select `es-ES-ElviraNeural`, the text is spoken in English with a Spanish accent.

If the voice doesn't speak the language of the input text, the Speech service doesn't create synthesized audio. For a full list of supported neural voices, see Language and voice support for the Speech service.

> **Note**

The default voice is the first voice returned per locale from the Voice List API.

The voice that speaks is determined in order of priority as follows:

- If you don't set `SpeechSynthesisVoiceName` or `SpeechSynthesisLanguage`, the default voice for `en-US` speaks.
- If you only set `SpeechSynthesisLanguage`, the default voice for the specified locale speaks.
- If both `SpeechSynthesisVoiceName` and `SpeechSynthesisLanguage` are set, the `SpeechSynthesisLanguage` setting is ignored. The voice that you specify by using `SpeechSynthesisVoiceName` speaks.
- If the voice element is set by using Speech Synthesis Markup Language (SSML), the `SpeechSynthesisVoiceName` and `SpeechSynthesisLanguage` settings are ignored.

In summary, the order of priority can be described as:

Expand table

`SpeechSynthesisVoiceName`	`SpeechSynthesisLanguage`	SSML	Outcome
X	X	X	Default voice for `en-US` speaks
X	✓	X	Default voice for specified locale speaks.
✓	✓	X	The voice that you specify by using `SpeechSynthesisVoiceName` speaks.
✓	✓	✓	The voice that you specify by using SSML speaks.

Synthesize speech to a file

Create a SpeechSynthesizer object. This object shown in the following snippets runs text to speech conversions and outputs to speakers, files, or other output streams. `SpeechSynthesizer` accepts as parameters:

- The SpeechConfig object that you created in the previous step.
- An AudioConfig object that specifies how output results should be handled.

1. Create an `AudioConfig` instance to automatically write the output to a *.wav* file by using the `FromWavFileOutput()` function. Instantiate it with a `using` statement.

   C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");
    using var audioConfig =
AudioConfig.FromWavFileOutput("path/to/write/file.wav");
}
```

A `using` statement in this context automatically disposes of unmanaged resources and causes the object to go out of scope after disposal.

2. Instantiate a `SpeechSynthesizer` instance with another `using` statement. Pass your `speechConfig` object and the `audioConfig` object as parameters. To synthesize speech and write to a file, run `SpeakTextAsync()` with a string of text.

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");
    using var audioConfig =
AudioConfig.FromWavFileOutput("path/to/write/file.wav");
    using var speechSynthesizer = new SpeechSynthesizer(speechConfig,
audioConfig);
    await speechSynthesizer.SpeakTextAsync("I'm excited to try text to speech");
}
```

When you run the program, it creates a synthesized *.wav* file, which is written to the location that you specify. This result is a good example of the most basic usage. Next, you can customize output and handle the output response as an in-memory stream for working with custom scenarios.

Synthesize to speaker output

To output synthesized speech to the current active output device such as a speaker, omit the `AudioConfig` parameter when you're creating the `SpeechSynthesizer` instance. Here's an example:

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");
```

```
    using var speechSynthesizer = new SpeechSynthesizer(speechConfig);
    await speechSynthesizer.SpeakTextAsync("I'm excited to try text to speech");
}
```

Get a result as an in-memory stream

You can use the resulting audio data as an in-memory stream rather than directly writing to a file. With in-memory stream, you can build custom behavior:

- Abstract the resulting byte array as a seekable stream for custom downstream services.
- Integrate the result with other APIs or services.
- Modify the audio data, write custom .wav headers, and do related tasks.

You can make this change to the previous example. First, remove the `AudioConfig` block, because you manage the output behavior manually from this point onward for increased control. Pass `null` for `AudioConfig` in the `SpeechSynthesizer` constructor.

> **Note**
>
> Passing `null` for `AudioConfig`, rather than omitting it as in the previous speaker output example, doesn't play the audio by default on the current active output device.

Save the result to a SpeechSynthesisResult variable. The `AudioData` property contains a `byte []` instance for the output data. You can work with this `byte []` instance manually, or you can use the AudioDataStream class to manage the in-memory stream.

In this example, you use the `AudioDataStream.FromResult()` static function to get a stream from the result:

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");
    using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);

    var result = await speechSynthesizer.SpeakTextAsync("I'm excited to try text
to speech");
    using var stream = AudioDataStream.FromResult(result);
}
```

At this point, you can implement any custom behavior by using the resulting `stream` object.

Customize audio format

You can customize audio output attributes, including:

- Audio file type
- Sample rate
- Bit depth

To change the audio format, you use the `SetSpeechSynthesisOutputFormat()` function on the `SpeechConfig` object. This function expects an `enum` instance of type SpeechSynthesisOutputFormat. Use the `enum` to select the output format. For available formats, see the list of audio formats.

There are various options for different file types, depending on your requirements. By definition, raw formats like `Raw24Khz16BitMonoPcm` don't include audio headers. Use raw formats only in one of these situations:

- You know that your downstream implementation can decode a raw bitstream.
- You plan to manually build headers based on factors like bit depth, sample rate, and number of channels.

This example specifies the high-fidelity RIFF format `Riff24Khz16BitMonoPcm` by setting `SpeechSynthesisOutputFormat` on the `SpeechConfig` object. Similar to the example in the previous section, you use AudioDataStream to get an in-memory stream of the result, and then write it to a file.

C#

```
static async Task SynthesizeAudioAsync()
{
    var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
"YourSpeechRegion");

speechConfig.SetSpeechSynthesisOutputFormat(SpeechSynthesisOutputFormat.Riff24Khz1
6BitMonoPcm);

    using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);
    var result = await speechSynthesizer.SpeakTextAsync("I'm excited to try text
to speech");

    using var stream = AudioDataStream.FromResult(result);
    await stream.SaveToWaveFileAsync("path/to/write/file.wav");
}
```

When you run the program, it writes a *.wav* file to the specified path.

Use SSML to customize speech characteristics

You can use SSML to fine-tune the pitch, pronunciation, speaking rate, volume, and other aspects in the text to speech output by submitting your requests from an XML schema. This section shows an example of changing the voice. For more information, see Speech Synthesis Markup Language overview.

To start using SSML for customization, you make a minor change that switches the voice.

1. Create a new XML file for the SSML configuration in your root project directory.

   XML

   ```xml
   <speak version="1.0" xmlns="https://www.w3.org/2001/10/synthesis"
   xml:lang="en-US">
     <voice name="en-US-AvaMultilingualNeural">
       When you're on the freeway, it's a good idea to use a GPS.
     </voice>
   </speak>
   ```

   In this example, the file is *ssml.xml*. The root element is always `<speak>`. Wrapping the text in a `<voice>` element allows you to change the voice by using the `name` parameter. For the full list of supported neural voices, see Supported languages.

2. Change the speech synthesis request to reference your XML file. The request is mostly the same, but instead of using the `SpeakTextAsync()` function, you use `SpeakSsmlAsync()`. This function expects an XML string. First, load your SSML configuration as a string by using `File.ReadAllText()`. From this point, the result object is exactly the same as previous examples.

   > **Note**
   >
   > If you're using Visual Studio, your build configuration likely won't find your XML file by default. Right-click the XML file and select **Properties**. Change **Build Action** to **Content**. Change **Copy to Output Directory** to **Copy always**.

   C#

   ```csharp
   public static async Task SynthesizeAudioAsync()
   {
       var speechConfig = SpeechConfig.FromSubscription("YourSpeechKey",
   "YourSpeechRegion");
       using var speechSynthesizer = new SpeechSynthesizer(speechConfig, null);

       var ssml = File.ReadAllText("./ssml.xml");
   ```

```
    var result = await speechSynthesizer.SpeakSsmlAsync(ssml);

    using var stream = AudioDataStream.FromResult(result);
    await stream.SaveToWaveFileAsync("path/to/write/file.wav");
}
```

> **Note**
>
> To change the voice without using SSML, you can set the property on `SpeechConfig` by using `SpeechConfig.SpeechSynthesisVoiceName = "en-US-AvaMultilingualNeural";`.

Subscribe to synthesizer events

You might want more insights about the text to speech processing and results. For example, you might want to know when the synthesizer starts and stops, or you might want to know about other events encountered during synthesis.

While using the SpeechSynthesizer for text to speech, you can subscribe to the events in this table:

Expand table

Event	Description	Use case
`BookmarkReached`	Signals that a bookmark was reached. To trigger a bookmark reached event, a `bookmark` element is required in the SSML. This event reports the output audio's elapsed time between the beginning of synthesis and the `bookmark` element. The event's `Text` property is the string value that you set in the bookmark's `mark` attribute. The `bookmark` elements aren't spoken.	You can use the `bookmark` element to insert custom markers in SSML to get the offset of each marker in the audio stream. The `bookmark` element can be used to reference a specific location in the text or tag sequence.
`SynthesisCanceled`	Signals that the speech synthesis was canceled.	You can confirm when synthesis is canceled.
`SynthesisCompleted`	Signals that speech synthesis is complete.	You can confirm when synthesis is complete.
`SynthesisStarted`	Signals that speech synthesis started.	You can confirm when synthesis started.

Event	Description	Use case
`Synthesizing`	Signals that speech synthesis is ongoing. This event fires each time the SDK receives an audio chunk from the Speech service.	You can confirm when synthesis is in progress.
`VisemeReceived`	Signals that a viseme event was received.	Visemes are often used to represent the key poses in observed speech. Key poses include the position of the lips, jaw, and tongue in producing a particular phoneme. You can use visemes to animate the face of a character as speech audio plays.
`WordBoundary`	Signals that a word boundary was received. This event is raised at the beginning of each new spoken word, punctuation, and sentence. The event reports the current word's time offset, in ticks, from the beginning of the output audio. This event also reports the character position in the input text or SSML immediately before the word that's about to be spoken.	This event is commonly used to get relative positions of the text and corresponding audio. You might want to know about a new word, and then take action based on the timing. For example, you can get information that can help you decide when and for how long to highlight words as they're spoken.

> **Note**
>
> Events are raised as the output audio data becomes available, which is faster than playback to an output device. The caller must appropriately synchronize streaming and real-time.

Here's an example that shows how to subscribe to events for speech synthesis.

> **Important**
>
> Use API keys with caution. Don't include the API key directly in your code, and never post it publicly. If you use an API key, store it securely in Azure Key Vault. For more information about using API keys securely in your apps, see API keys with Azure Key Vault.
>
> For more information about AI services security, see Authenticate requests to Azure AI services.

You can follow the instructions in the quickstart, but replace the contents of that *Program.cs* file with the following C# code:

C#

```
using Microsoft.CognitiveServices.Speech;

class Program
{
    // This example requires environment variables named "SPEECH_KEY" and
"SPEECH_REGION"
    static string speechKey = Environment.GetEnvironmentVariable("SPEECH_KEY");
    static string speechRegion =
Environment.GetEnvironmentVariable("SPEECH_REGION");

    async static Task Main(string[] args)
    {
        var speechConfig = SpeechConfig.FromSubscription(speechKey, speechRegion);

        var speechSynthesisVoiceName  = "en-US-AvaMultilingualNeural";
        var ssml = @$"<speak version='1.0' xml:lang='en-US'
xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='http://www.w3.org/2001/mstts'>
            <voice name='{speechSynthesisVoiceName}'>
                <mstts:viseme type='redlips_front'/>
                The rainbow has seven colors: <bookmark
mark='colors_list_begin'/>Red, orange, yellow, green, blue, indigo, and violet.
<bookmark mark='colors_list_end'/>.
            </voice>
        </speak>";

        // Required for sentence-level WordBoundary events

speechConfig.SetProperty(PropertyId.SpeechServiceResponse_RequestSentenceBoundary,
"true");

        using (var speechSynthesizer = new SpeechSynthesizer(speechConfig))
        {
            // Subscribe to events

            speechSynthesizer.BookmarkReached += (s, e) =>
            {
                Console.WriteLine($"BookmarkReached event:" +
                    $"\r\n\tAudioOffset: {(e.AudioOffset + 5000) / 10000}ms" +
                    $"\r\n\tText: \"{e.Text}\".");
            };

            speechSynthesizer.SynthesisCanceled += (s, e) =>
            {
                Console.WriteLine("SynthesisCanceled event");
            };

            speechSynthesizer.SynthesisCompleted += (s, e) =>
            {
                Console.WriteLine($"SynthesisCompleted event:" +
                    $"\r\n\tAudioData: {e.Result.AudioData.Length} bytes" +
                    $"\r\n\tAudioDuration: {e.Result.AudioDuration}");
```

```
            };

            speechSynthesizer.SynthesisStarted += (s, e) =>
            {
                Console.WriteLine("SynthesisStarted event");
            };

            speechSynthesizer.Synthesizing += (s, e) =>
            {
                Console.WriteLine($"Synthesizing event:" +
                    $"\r\n\tAudioData: {e.Result.AudioData.Length} bytes");
            };

            speechSynthesizer.VisemeReceived += (s, e) =>
            {
                Console.WriteLine($"VisemeReceived event:" +
                    $"\r\n\tAudioOffset: {(e.AudioOffset + 5000) / 10000}ms" +
                    $"\r\n\tVisemeId: {e.VisemeId}");
            };

            speechSynthesizer.WordBoundary += (s, e) =>
            {
                Console.WriteLine($"WordBoundary event:" +
                    // Word, Punctuation, or Sentence
                    $"\r\n\tBoundaryType: {e.BoundaryType}" +
                    $"\r\n\tAudioOffset: {(e.AudioOffset + 5000) / 10000}ms" +
                    $"\r\n\tDuration: {e.Duration}" +
                    $"\r\n\tText: \"{e.Text}\"" +
                    $"\r\n\tTextOffset: {e.TextOffset}" +
                    $"\r\n\tWordLength: {e.WordLength}");
            };

            // Synthesize the SSML
            Console.WriteLine($"SSML to synthesize: \r\n{ssml}");
            var speechSynthesisResult = await
speechSynthesizer.SpeakSsmlAsync(ssml);

            // Output the results
            switch (speechSynthesisResult.Reason)
            {
                case ResultReason.SynthesizingAudioCompleted:
                    Console.WriteLine("SynthesizingAudioCompleted result");
                    break;
                case ResultReason.Canceled:
                    var cancellation =
SpeechSynthesisCancellationDetails.FromResult(speechSynthesisResult);
                    Console.WriteLine($"CANCELED: Reason={cancellation.Reason}");

                    if (cancellation.Reason == CancellationReason.Error)
                    {
                        Console.WriteLine($"CANCELED: ErrorCode=
{cancellation.ErrorCode}");
                        Console.WriteLine($"CANCELED: ErrorDetails=
[{cancellation.ErrorDetails}]");
                        Console.WriteLine($"CANCELED: Did you set the speech
```

```
resource key and region values?");
                    }
                    break;
                default:
                    break;
            }
        }

        Console.WriteLine("Press any key to exit...");
        Console.ReadKey();
    }
}
```

You can find more text to speech samples at GitHub.

Use a custom endpoint

The custom endpoint is functionally identical to the standard endpoint used for text to speech requests.

One difference is that the `EndpointId` must be specified to use your custom voice via the Speech SDK. You can start with the text to speech quickstart and then update the code with the `EndpointId` and `SpeechSynthesisVoiceName`.

C#

```
var speechConfig = SpeechConfig.FromSubscription(speechKey, speechRegion);
speechConfig.SpeechSynthesisVoiceName = "YourCustomVoiceName";
speechConfig.EndpointId = "YourEndpointId";
```

To use a custom voice via Speech Synthesis Markup Language (SSML), specify the model name as the voice name. This example uses the `YourCustomVoiceName` voice.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="YourCustomVoiceName">
        This is the text that is spoken.
    </voice>
</speak>
```

Run and use a container

Speech containers provide websocket-based query endpoint APIs that are accessed through the Speech SDK and Speech CLI. By default, the Speech SDK and Speech CLI use the public

Speech service. To use the container, you need to change the initialization method. Use a container host URL instead of key and region.

For more information about containers, see Install and run Speech containers with Docker.

Next steps

- Try the text to speech quickstart
- Get started with custom voice
- Improve synthesis with SSML

Batch synthesis API for text to speech

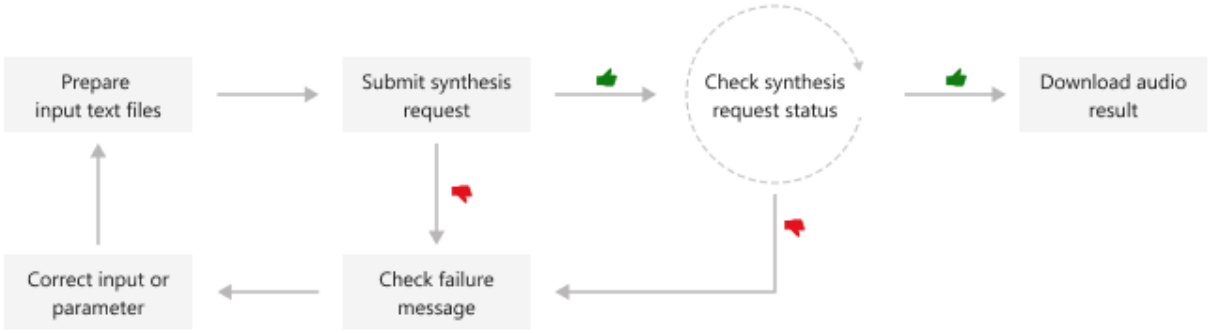
08/07/2025

The Batch synthesis API can synthesize a large volume of text input (long and short) asynchronously. Publishers and audio content platforms can create long audio content in a batch. For example: audio books, news articles, and documents. The batch synthesis API can create synthesized audio longer than 10 minutes.

> **Important**
>
> The Batch synthesis API is generally available. The Long Audio API will be retired on April 1st, 2027. For more information, see Migrate to batch synthesis API.

The batch synthesis API is asynchronous and doesn't return synthesized audio in real-time. You submit text files to be synthesized, poll for the status, and download the audio output when the status indicates success. The text inputs must be plain text or Speech Synthesis Markup Language (SSML) text.

This diagram provides a high-level overview of the workflow.



> **Tip**
>
> You can also use the Speech SDK to create synthesized audio longer than 10 minutes by iterating over the text and synthesizing it in chunks. For a C# example, see GitHub.

You can use the following REST API operations for batch synthesis:

Expand table

Operation	Method	REST API call
Create batch synthesis	`PUT`	texttospeech/batchsyntheses/YourSynthesisId
Get batch synthesis	`GET`	texttospeech/batchsyntheses/YourSynthesisId
List batch synthesis	`GET`	texttospeech/batchsyntheses
Delete batch synthesis	`DELETE`	texttospeech/batchsyntheses/YourSynthesisId

For code samples, see GitHub.

Create batch synthesis

To submit a batch synthesis request, construct the HTTP PUT request path and body according to the following instructions:

- Set the required `inputKind` property.
- If the `inputKind` property is set to "PlainText", then you must also set the `voice` property in the `synthesisConfig`. In the following example, the `inputKind` is set to "SSML", so the `synthesisConfig` isn't set.
- Optionally you can set the `description`, `timeToLiveInHours`, and other properties. For more information, see batch synthesis properties.

> **Note**
>
> The maximum JSON payload size that's accepted is 2 megabytes.

Set the required `YourSynthesisId` in path. The `YourSynthesisId` must be unique. It must be 3-64 long, contains only numbers, letters, hyphens, underscores and dots, starts and ends with a letter or number.

Make an HTTP PUT request using the URI as shown in the following example. Replace `YourSpeechKey` with your Speech resource key, replace `YourSpeechRegion` with your Speech resource region, and set the request body properties as previously described.

Azure CLI

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourSpeechKey" -H "Content-Type:
application/json" -d '{
    "description": "my ssml test",
    "inputKind": "SSML",
    "inputs": [
        {
```

```
            "content": "<speak version=\"1.0\" xml:lang=\"en-US\"><voice
name=\"en-US-JennyNeural\">The rainbow has seven colors.</voice></speak>"
        }
    ],
    "properties": {
        "outputFormat": "riff-24khz-16bit-mono-pcm",
        "wordBoundaryEnabled": false,
        "sentenceBoundaryEnabled": false,
        "concatenateResult": false,
        "decompressOutputFiles": false
    }
}'
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/
YourSynthesisId?api-version=2024-04-01"
```

You should receive a response body in the following format:

JSON

```
{
  "id": "YourSynthesisId",
  "status": "Running",
  "createdDateTime": "2024-03-12T07:23:18.0097387Z",
  "lastActionDateTime": "2024-03-12T07:23:18.0097388Z",
  "inputKind": "SSML",
  "customVoices": {},
  "properties": {
    "timeToLiveInHours": 168,
    "outputFormat": "riff-24khz-16bit-mono-pcm",
    "concatenateResult": false,
    "decompressOutputFiles": false,
    "wordBoundaryEnabled": false,
    "sentenceBoundaryEnabled": false
  }
}
```

The `status` property should progress from `Running` status to `Succeeded` or `Failed`. You can call the GET batch synthesis API periodically until the returned status is `Succeeded` or `Failed`.

Get batch synthesis

To get the status of the batch synthesis job, make an HTTP GET request using the URI as shown in the following example. Replace `YourSpeechKey` with your Speech resource key, and replace `YourSpeechRegion` with your Speech resource region.

Azure CLI

```
curl -v -X GET
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/
YourSynthesisId?api-version=2024-04-01" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey"
```

You should receive a response body in the following format:

JSON


```
{
  "id": "YourSynthesisId",
  "status": "Succeeded",
  "createdDateTime": "2024-03-12T07:23:18.0097387Z",
  "lastActionDateTime": "2024-03-12T07:23:18.7979669",
  "inputKind": "SSML",
  "customVoices": {},
  "properties": {
    "timeToLiveInHours": 168,
    "outputFormat": "riff-24khz-16bit-mono-pcm",
    "concatenateResult": false,
    "decompressOutputFiles": false,
    "wordBoundaryEnabled": false,
    "sentenceBoundaryEnabled": false,
    "sizeInBytes": 120000,
    "succeededAudioCount": 1,
    "failedAudioCount": 0,
    "durationInMilliseconds": 2500,
    "billingDetails": {
      "neuralCharacters": 29
    }
  },
  "outputs": {
    "result": "https://stttssvcuse.blob.core.windows.net/batchsynthesis-
output/29f2105f997c4bfea176d39d05ff201e/YourSynthesisId/results.zip?SAS_Token"
  }
}
```


From `outputs.result`, you can download a ZIP file that contains the audio (such as `0001.wav`), summary, and debug details. For more information, see batch synthesis results.

List batch synthesis

To list all batch synthesis jobs for the Speech resource, make an HTTP GET request using the URI as shown in the following example. Replace `YourSpeechKey` with your Speech resource key and replace `YourSpeechRegion` with your Speech resource region. Optionally, you can set the `skip` and `maxpagesize` (up to 100) query parameters in URL. The default value for `skip` is 0 and the default value for `maxpagesize` is 100.

Azure CLI

```
curl -v -X GET
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses?
api-version=2024-04-01&skip=1&maxpagesize=2" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey"
```

You should receive a response body in the following format:

JSON


```
{
  "value": [
    {
      "id": "my-job-03",
      "status": "Succeeded",
      "createdDateTime": "2024-03-12T07:28:32.5690441Z",
      "lastActionDateTime": "2024-03-12T07:28:33.0042293",
      "inputKind": "SSML",
      "customVoices": {},
      "properties": {
        "timeToLiveInHours": 168,
        "outputFormat": "riff-24khz-16bit-mono-pcm",
        "concatenateResult": false,
        "decompressOutputFiles": false,
        "wordBoundaryEnabled": false,
        "sentenceBoundaryEnabled": false,
        "sizeInBytes": 120000,
        "succeededAudioCount": 1,
        "failedAudioCount": 0,
        "durationInMilliseconds": 2500,
        "billingDetails": {
          "neuralCharacters": 29
        }
      },
      "outputs": {
        "result": "https://stttssvcuse.blob.core.windows.net/batchsynthesis-
output/29f2105f997c4bfea176d39d05ff201e/my-job-03/results.zip?SAS_Token"
      }
    },
    {
      "id": "my-job-02",
      "status": "Succeeded",
      "createdDateTime": "2024-03-12T07:28:29.6418211Z",
      "lastActionDateTime": "2024-03-12T07:28:30.0910306",
      "inputKind": "SSML",
      "customVoices": {},
      "properties": {
        "timeToLiveInHours": 168,
        "outputFormat": "riff-24khz-16bit-mono-pcm",
        "concatenateResult": false,
        "decompressOutputFiles": false,
        "wordBoundaryEnabled": false,
```

```
        "sentenceBoundaryEnabled": false,
        "sizeInBytes": 120000,
        "succeededAudioCount": 1,
        "failedAudioCount": 0,
        "durationInMilliseconds": 2500,
        "billingDetails": {
          "neuralCharacters": 29
        }
      },
      "outputs": {
        "result": "https://stttssvcuse.blob.core.windows.net/batchsynthesis-
output/29f2105f997c4bfea176d39d05ff201e/my-job-02/results.zip?SAS_Token"
      }
    }
  ],
  "nextLink":
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses?
skip=3&maxpagesize=2&api-version=2024-04-01"
}
```


From `outputs.result`, you can download a ZIP file that contains the audio (such as `0001.wav`), summary, and debug details. For more information, see batch synthesis results.

The `value` property in the json response lists your synthesis requests. The list is paginated, with a maximum page size of 100. The `"nextLink"` property is provided as needed to get the next page of the paginated list.

Delete batch synthesis

Delete the batch synthesis job history after you retrieved the audio output results. The Speech service keeps batch synthesis history for 168 hours (7 days) by default. Alternatively, you can specify this retention period using the `timeToLiveInHours` property, up to 744 hours (31 days). The date and time of automatic deletion (for synthesis jobs with a status of "Succeeded" or "Failed") is equal to the `lastActionDateTime` + `timeToLiveInHours` properties.

To delete a batch synthesis job, make an HTTP DELETE request using the URI as shown in the following example. Replace `YourSynthesisId` with your batch synthesis ID, replace `YourSpeechKey` with your Speech resource key, and replace `YourSpeechRegion` with your Speech resource region.

Azure CLI

```
curl -v -X DELETE
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/
YourSynthesisId?api-version=2024-04-01" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey"
```

The response headers include `HTTP/1.1 204 No Content` if the delete request was successful.

Batch synthesis results

After you get a batch synthesis job with `status` of "Succeeded", you can download the audio output results. Use the URL from the `outputs.result` property of the batch synthesis GET response.

To get the batch synthesis results file, make an HTTP GET request using the URI as shown in the following example. Replace `YourOutputsResultUrl` with the URL from the `outputs.result` property of the batch synthesis GET response. Replace `YourSpeechKey` with your Speech resource key.

Azure CLI

```
curl -v -X GET "YourOutputsResultUrl" -H "Ocp-Apim-Subscription-Key:
YourSpeechKey" > results.zip
```

The results are in a ZIP file that contains the audio (such as `0001.wav`), summary, and debug details. The numbered prefix of each filename (shown below as `[nnnn]`) is in the same order as the text inputs used when you created the batch synthesis.

> **Note**
>
> The `[nnnn].debug.json` file contains the synthesis result ID and other information that might help with troubleshooting. The properties that it contains might change, so you shouldn't take any dependencies on the JSON format.

The summary file contains the synthesis results for each text input. Here's an example `summary.json` file:

JSON

```
{
  "jobID": "7ab84171-9070-4d3b-88d4-1b8cc1cb928a",
  "status": "Succeeded",
  "results": [
    {
      "contents": ["<speak version=\"1.0\" xml:lang=\"en-US\"><voice name=\"en-US-
JennyNeural\">The rainbow has seven colors.</voice></speak>"],
      "status": "Succeeded",
      "audioFileName": "0001.wav",
      "properties": {
        "sizeInBytes": "120000",
```

```
        "durationInMilliseconds": "2500"
      }
    }
  ]
}
```

If sentence boundary data was requested (`"sentenceBoundaryEnabled": true`), then a corresponding `[nnnn].sentence.json` file is included in the results. Likewise, if word boundary data was requested (`"wordBoundaryEnabled": true`), then a corresponding `[nnnn].word.json` file is included in the results.

Here's an example word data file with both audio offset and duration in milliseconds:

JSON

```
[
  {
    "Text": "The",
    "AudioOffset": 50,
    "Duration": 137
  },
  {
    "Text": "rainbow",
    "AudioOffset": 200,
    "Duration": 350
  },
  {
    "Text": "has",
    "AudioOffset": 562,
    "Duration": 175
  },
  {
    "Text": "seven",
    "AudioOffset": 750,
    "Duration": 300
  },
  {
    "Text": "colors",
    "AudioOffset": 1062,
    "Duration": 625
  },
  {
    "Text": ".",
    "AudioOffset": 1700,
    "Duration": 100
  }
]
```

Batch synthesis latency and best practices

When using batch synthesis for generating synthesized speech, it's important to consider the latency involved and follow best practices for achieving optimal results.

Latency in batch synthesis

The latency in batch synthesis depends on various factors, including the complexity of the input text, the number of inputs in the batch, and the processing capabilities of the underlying hardware.

The latency for batch synthesis is as follows (approximately):

- The latency of 50% of the synthesized speech outputs is within 10-20 seconds.
- The latency of 95% of the synthesized speech outputs is within 120 seconds.

Best practices

When considering batch synthesis for your application, it's recommended to assess whether the latency meets your requirements. If the latency aligns with your desired performance, batch synthesis can be a suitable choice. However, if the latency doesn't meet your needs, you might consider using real-time API.

HTTP status codes

The section details the HTTP response codes and messages from the batch synthesis API.

HTTP 200 OK

HTTP 200 OK indicates that the request was successful.

HTTP 201 Created

HTTP 201 Created indicates that the batch synthesis create request (via HTTP PUT) was successful.

HTTP 204 error

An HTTP 204 error indicates that the request was successful, but the resource doesn't exist. For example:

- You tried to get or delete a synthesis job that doesn't exist.

- You successfully deleted a synthesis job.

HTTP 400 error

Here are examples that can result in the 400 error:

- The `outputFormat` is unsupported or invalid. Provide a valid format value, or leave `outputFormat` empty to use the default setting.
- The number of requested text inputs exceeded the limit of 10,000.
- You tried to use an invalid deployment ID or a custom voice that isn't successfully deployed. Make sure the Speech resource has access to the custom voice, and the custom voice is successfully deployed. You must also ensure that the mapping of `{"your-custom-voice-name": "your-deployment-ID"}` is correct in your batch synthesis request.
- You tried to use a *F0* Speech resource, but the region only supports the *Standard* Speech resource pricing tier.

HTTP 404 error

The specified entity can't be found. Make sure the synthesis ID is correct.

HTTP 429 error

There are too many recent requests. Each client application can submit up to 100 requests per 10 seconds for each Speech resource. Reduce the number of requests per second.

HTTP 500 error

HTTP 500 Internal Server Error indicates that the request failed. The response body contains the error message.

HTTP error example

Here's an example request that results in an HTTP 400 error, because the `inputs` property is required to create a job.

Console

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourSpeechKey" -H "Content-Type:
application/json" -d '{
    "inputKind": "SSML"
}'
```

```
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/
YourSynthesisId?api-version=2024-04-01"
```

In this case, the response headers include `HTTP/1.1 400 Bad Request`.

The response body resembles the following JSON example:

JSON

```
{
  "error": {
    "code": "BadRequest",
    "message": "The inputs is required."
  }
}
```

Next steps

- Speech Synthesis Markup Language (SSML)
- Batch synthesis properties
- Migrate to batch synthesis

Batch synthesis properties for text to speech

08/07/2025

> **Important**
>
> The Batch synthesis API is generally available. The Long Audio API will be retired on April 1st, 2027. For more information, see Migrate to batch synthesis API.

The Batch synthesis API can synthesize a large volume of text input (long and short) asynchronously. Publishers and audio content platforms can create long audio content in a batch. For example: audio books, news articles, and documents. The batch synthesis API can create synthesized audio longer than 10 minutes.

Some properties in JSON format are required when you create a new batch synthesis job. Other properties are optional. The batch synthesis response includes other properties to provide information about the synthesis status and results. For example, the `outputs.result` property contains the location of the batch synthesis result files with audio output and logs.

Batch synthesis properties

Batch synthesis properties are described in the following table.

Expand table

Property	Description
`createdDateTime`	The date and time when the batch synthesis job was created. This property is read-only.
`customVoices`	The map of a custom voice name and its deployment ID. For example: `"customVoices": {"your-custom-voice-name": "502ac834-6537-4bc3-9fd6-140114daa66d"}` You can use the voice name in your `synthesisConfig.voice` (when the `inputKind` is set to `"PlainText"`) or within the SSML text of `inputs` (when the `inputKind` is set to `"SSML"`). This property is required to use a custom voice. If you try to

Property	Description
	use a custom voice that isn't defined here, the service returns an error.
`description`	The description of the batch synthesis. This property is optional.
`id`	The batch synthesis job ID you passed in path. This property is required in path.
`inputs`	The plain text or SSML to be synthesized. When the `inputKind` is set to `"PlainText"`, provide plain text as shown here: `"inputs": [{"content": "The rainbow has seven colors."}]`. When the `inputKind` is set to `"SSML"`, provide text in the Speech Synthesis Markup Language (SSML) as shown here: `"inputs": [{"content": "<speak version='1.0' xml:lang='en-US'><voice xml:lang='en-US' xml:gender='Female' name='en-US-AvaMultilingualNeural'>The rainbow has seven colors. </voice></speak>"}]`. Include up to 1,000 text objects if you want multiple audio output files. Here's example input text that should be synthesized to two audio output files: `"inputs": [{"content": "synthesize this to a file"},{"content": "synthesize this to another file"}]`. However, if the `properties.concatenateResult` property is set to `true`, then each synthesized result is written to the same audio output file. You don't need separate text inputs for new paragraphs. Within any of the (up to 1,000) text inputs, you can specify new paragraphs using the "\r\n" (newline) string. Here's example input text with two paragraphs that should be synthesized to the same audio output file: `"inputs": [{"content": "synthesize this to a file\r\nsynthesize this to another paragraph in the same file"}]` There are no paragraph limits, but the maximum JSON payload size (including all text inputs and other properties) is 2 megabytes. This property is required when you create a new batch synthesis job. This property isn't included in the response when you get the synthesis job.

Property	Description
`lastActionDateTime`	The most recent date and time when the `status` property value changed. This property is read-only.
`outputs.result`	The location of the batch synthesis result files with audio output and logs. This property is read-only.
`properties`	A defined set of optional batch synthesis configuration settings.
`properties.sizeInBytes`	The audio output size in bytes. This property is read-only.
`properties.billingDetails`	The number of words that were processed and billed by `customNeuralCharacters` (custom voice) versus `neuralCharacters` (standard voice). This property is read-only.
`properties.concatenateResult`	Determines whether to concatenate the result. This optional `bool` value ("true" or "false") is "false" by default.
`properties.decompressOutputFiles`	Determines whether to unzip the synthesis result files in the destination container. This property can only be set when the `destinationContainerUrl` property is set. This optional `bool` value ("true" or "false") is "false" by default.
`properties.destinationContainerUrl`	The batch synthesis results can be stored in a writable Azure container. If you don't specify a container URI with shared access signatures (SAS) token, the Speech service stores the results in a container managed by Microsoft. SAS with stored access policies isn't supported. When the synthesis job is deleted, the result data is also deleted. This optional property isn't included in the response when you get the synthesis job.
`properties.destinationPath`	The prefix path for storing batch synthesis results. If no prefix path is provided, a system-generated path will be used. This property is optional and can only be set when the `destinationContainerUrl` property is specified.
`properties.durationInMilliseconds`	The audio output duration in milliseconds.

Property	Description
	This property is read-only.
`properties.failedAudioCount`	The count of batch synthesis inputs to audio output failed. This property is read-only.
`properties.outputFormat`	The audio output format. For information about the accepted values, see audio output formats. The default output format is `riff-24khz-16bit-mono-pcm`.
`properties.sentenceBoundaryEnabled`	Determines whether to generate sentence boundary data. This optional `bool` value ("true" or "false") is "false" by default. If sentence boundary data is requested, then a corresponding `[nnnn].sentence.json` file is included in the results data ZIP file.
`properties.succeededAudioCount`	The count of batch synthesis inputs to audio output succeeded. This property is read-only.
`properties.timeToLiveInHours`	A duration in hours after the synthesis job is completed, when the synthesis results will be automatically deleted. This optional setting is `168` (7 days) by default. The maximum time to live is `744` (31 days). The date and time of automatic deletion (for synthesis jobs with a status of "Succeeded" or "Failed") is equal to the `lastActionDateTime` + `timeToLiveInHours` properties. Otherwise, you can call the delete synthesis method to remove the job sooner.
`properties.wordBoundaryEnabled`	Determines whether to generate word boundary data. This optional `bool` value ("true" or "false") is "false" by default. If word boundary data is requested, then a corresponding `[nnnn].word.json` file is included in the results data ZIP file.
`status`	The batch synthesis processing status. The status should progress from "Running" to either "Succeeded" or "Failed". This property is read-only.

Property	Description
`synthesisConfig`	The configuration settings to use for batch synthesis of plain text. This property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.backgroundAudio`	The background audio for each audio output. This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.backgroundAudio.fadein`	The duration of the background audio fade-in as milliseconds. The default value is `0`, which is the equivalent to no fade in. Accepted values: `0` to `10000` inclusive. For information, see the attributes table under add background audio in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.backgroundAudio.fadeout`	The duration of the background audio fade-out in milliseconds. The default value is `0`, which is the equivalent to no fade out. Accepted values: `0` to `10000` inclusive. For information, see the attributes table under add background audio in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.backgroundAudio.src`	The URI location of the background audio file. For information, see the attributes table under add background audio in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This property is required when `synthesisConfig.backgroundAudio` is set.
`synthesisConfig.backgroundAudio.volume`	The volume of the background audio file. Accepted values: `0` to `100` inclusive. The default value is `1`. For information, see the attributes table under add background audio in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored.

Property	Description
	This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.pitch`	The pitch of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.rate`	The rate of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.role`	For some voices, you can adjust the speaking role-play. The voice can imitate a different age and gender, but the voice name isn't changed. For example, a male voice can raise the pitch and change the intonation to imitate a female voice, but the voice name isn't changed. If the role is missing or isn't supported for your voice, this attribute is ignored. For information about the available styles per voice, see voice styles and roles. This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.speakerProfileId`	The speaker profile ID of a personal voice. For information about available personal voice base model names, see integrate personal voice. For information about how to get the speaker profile ID, see language and voice support. This property is required when `inputKind` is set to `"PlainText"`.
`synthesisConfig.style`	For some voices, you can adjust the speaking style to express different emotions like cheerfulness, empathy, and calm. You can optimize the voice for different scenarios like customer

Property	Description
	service, newscast, and voice assistant. For information about the available styles per voice, see voice styles and roles. This optional property is only applicable when `synthesisConfig.style` is set.
`synthesisConfig.styleDegree`	The intensity of the speaking style. You can specify a stronger or softer style to make the speech more expressive or subdued. The range of accepted values are: 0.01 to 2 inclusive. The default value is 1, which means the predefined style intensity. The minimum unit is 0.01, which results in a slight tendency for the target style. A value of 2 results in a doubling of the default style intensity. If the style degree is missing or isn't supported for your voice, this attribute is ignored. For information about the available styles per voice, see voice styles and roles. This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`synthesisConfig.voice`	The voice that speaks the audio output. For information about the available standard voices, see language and voice support. To use a custom voice, you must specify a valid custom voice and deployment ID mapping in the `customVoices` property. To use a personal voice, you need to specify the `synthesisConfig.speakerProfileId` property. This property is required when `inputKind` is set to `"PlainText"`.
`synthesisConfig.volume`	The volume of the audio output. For information about the accepted values, see the adjust prosody table in the Speech Synthesis Markup Language (SSML) documentation. Invalid values are ignored. This optional property is only applicable when `inputKind` is set to `"PlainText"`.
`inputKind`	Indicates whether the `inputs` text property should be plain text or SSML. The possible case-insensitive values are "PlainText" and "SSML". When the `inputKind` is set to

Property	Description
	`"PlainText"`, you must also set the `synthesisConfig` voice property. This property is required.

Batch synthesis latency and best practices

When using batch synthesis for generating synthesized speech, it's important to consider the latency involved and follow best practices for achieving optimal results.

Latency in batch synthesis

The latency in batch synthesis depends on various factors, including the complexity of the input text, the number of inputs in the batch, and the processing capabilities of the underlying hardware.

The latency for batch synthesis is as follows (approximately):

- The latency of 50% of the synthesized speech outputs is within 10-20 seconds.
- The latency of 95% of the synthesized speech outputs is within 120 seconds.

Best practices

When considering batch synthesis for your application, it's recommended to assess whether the latency meets your requirements. If the latency aligns with your desired performance, batch synthesis can be a suitable choice. However, if the latency doesn't meet your needs, you might consider using real-time API.

HTTP status codes

The section details the HTTP response codes and messages from the batch synthesis API.

HTTP 200 OK

HTTP 200 OK indicates that the request was successful.

HTTP 201 Created

HTTP 201 Created indicates that the create batch synthesis request (via HTTP POST) was successful.

HTTP 204 error

An HTTP 204 error indicates that the request was successful, but the resource doesn't exist. For example:

- You tried to get or delete a synthesis job that doesn't exist.
- You successfully deleted a synthesis job.

HTTP 400 error

Here are examples that can result in the 400 error:

- The `outputFormat` is unsupported or invalid. Provide a valid format value, or leave `outputFormat` empty to use the default setting.
- The number of requested text inputs exceeded the limit of 10,000.
- You tried to use an invalid deployment ID or a custom voice that isn't successfully deployed. Make sure the Speech resource has access to the custom voice, and the custom voice is successfully deployed. You must also ensure that the mapping of `{"your-custom-voice-name": "your-deployment-ID"}` is correct in your batch synthesis request.
- You tried to use a *F0* Speech resource, but the region only supports the *Standard* Speech resource pricing tier.

HTTP 404 error

The specified entity can't be found. Make sure the synthesis ID is correct.

HTTP 429 error

There are too many recent requests. Each client application can submit up to 100 requests per 10 seconds for each Speech resource. Reduce the number of requests per second.

HTTP 500 error

HTTP 500 Internal Server Error indicates that the request failed. The response body contains the error message.

HTTP error example

Here's an example request that results in an HTTP 400 error, because the `inputs` property is required to create a job.

Console

```
curl -v -X PUT -H "Ocp-Apim-Subscription-Key: YourSpeechKey" -H "Content-Type:
application/json" -d '{
    "inputKind": "SSML"
}'
"https://YourSpeechRegion.api.cognitive.microsoft.com/texttospeech/batchsyntheses/
YourSynthesisId?api-version=2024-04-01"
```

In this case, the response headers include `HTTP/1.1 400 Bad Request`.

The response body resembles the following JSON example:

JSON

```
{
  "error": {
    "code": "BadRequest",
    "message": "The inputs is required."
  }
}
```

Next steps

- Speech Synthesis Markup Language (SSML)
- Text to speech quickstart
- Migrate to batch synthesis

Speech Synthesis Markup Language (SSML) overview

08/07/2025

Speech Synthesis Markup Language (SSML) is an XML-based markup language that you can use to fine-tune your text to speech output attributes such as pitch, pronunciation, speaking rate, volume, and more. It gives you more control and flexibility than plain text input.

> **Tip**
>
> You can hear voices in different styles and pitches reading example text by using the **Voice Gallery**.

Use case scenarios

SSML is designed to give you flexibility in how you want your speech output to sound, and it provides different properties for how you can customize that output. You can use SSML to:

- Define the input text structure that determines the structure, content, and other characteristics of your text to speech output. For example, you can use SSML to define a paragraph, a sentence, a break or a pause, or silence. You can wrap text with event tags, like a bookmark or viseme, that your application can process later. A viseme is the visual description of a phoneme, the individual speech sounds, in spoken language.
- Choose the voice, language, name, style, and role. You can use multiple voices in a single SSML document. You can also adjust the emphasis, speaking rate, pitch, and volume. SSML can also insert prerecorded audio, such as a sound effect or a musical note.
- Control pronunciation of the output audio. For example, you can use SSML with phonemes and a custom lexicon to improve pronunciation. You can also use SSML to define how a word or mathematical expression is pronounced.

Ways to work with SSML

SSML functionality is available in various tools that might fit your use case.

> **Important**
>
> You're billed for each character that's converted to speech, including punctuation. Although the SSML document itself isn't billable, the service counts optional elements that

you use to adjust how the text is converted to speech, like phonemes and pitch, as billable characters. For more information, see the **pricing note**.

You can use SSML in the following ways:

- The audio content creation tool lets you author plain text and SSML in Speech Studio. You can listen to the output audio and adjust the SSML to improve speech synthesis. For more information, see Speech synthesis with the Audio Content Creation tool.
- The batch synthesis API accepts SSML via the `inputs` property.
- The Speech CLI accepts SSML via the `spx synthesize --ssml SSML` command line argument.
- The Speech SDK accepts SSML via the "speak" SSML method across the different supported languages.

Next steps

- SSML document structure and events
- Voice and sound with SSML
- Pronunciation with SSML
- Language and voice support for the Speech service

SSML document structure and events

08/07/2025

The Speech Synthesis Markup Language (SSML) with input text determines the structure, content, and other characteristics of the text to speech output. For example, you can use SSML to define a paragraph, a sentence, a break or a pause, or silence. You can wrap text with event tags such as bookmark or viseme that can be processed later by your application.

Refer to the sections below for details about how to structure elements in the SSML document.

> **Note**
>
> In addition to Azure AI Speech neural (non HD) voices, you can also use Azure AI Speech high definition (HD) voices and Azure OpenAI neural (HD and non HD) voices. The HD voices provide a higher quality for more versatile scenarios.
>
> Some voices don't support all Speech Synthesis Markup Language (SSML) tags. This includes neural text to speech HD voices, personal voices, and embedded voices.

- For Azure AI Speech high definition (HD) voices, check the SSML support here.
- For personal voice, you can find the SSML support here.
- For embedded voices, check the SSML support here.

Document structure

The Speech service implementation of SSML is based on the World Wide Web Consortium's Speech Synthesis Markup Language Version 1.0. The elements supported by the Speech can differ from the W3C standard.

Each SSML document is created with SSML elements or tags. These elements are used to adjust the voice, style, pitch, prosody, volume, and more.

Here's a subset of the basic structure and syntax of an SSML document:

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="string">
    <mstts:backgroundaudio src="string" volume="string" fadein="string"
fadeout="string"/>
    <mstts:voiceconversion url="string"/>
    <voice name="string" effect="string">
        <audio src="string"></audio>
```

```
        <bookmark mark="string"/>
        <break strength="string" time="string" />
        <emphasis level="value"></emphasis>
        <lang xml:lang="string"></lang>
        <lexicon uri="string"/>
        <math xmlns="http://www.w3.org/1998/Math/MathML"></math>
        <mstts:audioduration value="string"/>
        <mstts:ttsembedding speakerProfileId="string"></mstts:ttsembedding>
        <mstts:express-as style="string" styledegree="value" role="string">
</mstts:express-as>
        <mstts:silence type="string" value="string"/>
        <mstts:viseme type="string"/>
        <p></p>
        <phoneme alphabet="string" ph="string"></phoneme>
        <prosody pitch="value" contour="value" range="value" rate="value"
volume="value"></prosody>
        <s></s>
        <say-as interpret-as="string" format="string" detail="string"></say-as>
        <sub alias="string"></sub>
    </voice>
</speak>
```

Some examples of contents that are allowed in each element are described in the following list:

- `audio`: The body of the `audio` element can contain plain text or SSML markup that's spoken if the audio file is unavailable or unplayable. The `audio` element can also contain text and the following elements: `audio`, `break`, `p`, `s`, `phoneme`, `prosody`, `say-as`, and `sub`.
- `bookmark`: This element can't contain text or any other elements.
- `break`: This element can't contain text or any other elements.
- `emphasis`: This element can contain text and the following elements: `audio`, `break`, `emphasis`, `lang`, `phoneme`, `prosody`, `say-as`, and `sub`.
- `lang`: This element can contain all other elements except `mstts:backgroundaudio`, `voice`, and `speak`.
- `lexicon`: This element can't contain text or any other elements.
- `math`: This element can only contain text and MathML elements.
- `mstts:audioduration`: This element can't contain text or any other elements.
- `mstts:backgroundaudio`: This element can't contain text or any other elements.
- `<mstts:voiceconversion>`: This element can't contain text or any other elements. It specifies the source audio URL for the voice conversion.
- `mstts:embedding`: This element can contain text and the following elements: `audio`, `break`, `emphasis`, `lang`, `phoneme`, `prosody`, `say-as`, and `sub`.
- `mstts:express-as`: This element can contain text and the following elements: `audio`, `break`, `emphasis`, `lang`, `phoneme`, `prosody`, `say-as`, and `sub`.
- `mstts:silence`: This element can't contain text or any other elements.

- `mstts:viseme`: This element can't contain text or any other elements.
- `p`: This element can contain text and the following elements: `audio`, `break`, `phoneme`, `prosody`, `say-as`, `sub`, `mstts:express-as`, and `s`.
- `phoneme`: This element can only contain text and no other elements.
- `prosody`: This element can contain text and the following elements: `audio`, `break`, `p`, `phoneme`, `prosody`, `say-as`, `sub`, and `s`.
- `s`: This element can contain text and the following elements: `audio`, `break`, `phoneme`, `prosody`, `say-as`, `mstts:express-as`, and `sub`.
- `say-as`: This element can only contain text and no other elements.
- `sub`: This element can only contain text and no other elements.
- `speak`: The root element of an SSML document. This element can contain the following elements: `mstts:backgroundaudio` and `voice`.
- `voice`: This element can contain all other elements except `mstts:backgroundaudio` and `speak`.

The Speech service automatically handles punctuation as appropriate, such as pausing after a period, or using the correct intonation when a sentence ends with a question mark.

Special characters

To use the characters `&`, `<`, and `>` within the SSML element's value or text, you must use the entity format. Specifically you must use `&` in place of `&`, use `<` in place of `<`, and use `>` in place of `>`. Otherwise the SSML isn't parsed correctly.

For example, specify `green & yellow` instead of `green & yellow`. The following SSML is parsed as expected:

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        My favorite colors are green &amp; yellow.
    </voice>
</speak>
```

Special characters such as quotation marks, apostrophes, and brackets, must be escaped. For more information, see Extensible Markup Language (XML) 1.0: Appendix D.

Double or single quotation marks must enclose the attribute values. For example, `<prosody volume="90">` and `<prosody volume='90'>` are well-formed, valid elements, but `<prosody volume=90>` isn't recognized.

Speak root element

The `speak` element contains information such as version, language, and the markup vocabulary definition. The `speak` element is the root element that's required for all SSML documents. You must specify the default language within the `speak` element, whether or not the language is adjusted elsewhere such as within the lang element.

Here's the syntax for the `speak` element:

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xml:lang="string"></speak>
```

Expand table

Attribute	Description	Required or optional
`version`	Indicates the version of the SSML specification used to interpret the document markup. The current version is "1.0".	Required
`xml:lang`	The language of the root document. The value can contain a language code such as `en` (English), or a locale such as `en-US` (English - United States).	Required
`xmlns`	The URI to the document that defines the markup vocabulary (the element types and attribute names) of the SSML document. The current URI is "http://www.w3.org/2001/10/synthesis".	Required

The `speak` element must contain at least one voice element.

speak examples

The supported values for attributes of the `speak` element were described previously.

Single voice example

This example uses the `en-US-AvaNeural` voice. For more examples, see voice examples.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        This is the text that is spoken.
```

```
    </voice>
</speak>
```

Add a break

Use the `break` element to override the default behavior of breaks or pauses between words. Otherwise the Speech service automatically inserts pauses.

Usage of the `break` element's attributes are described in the following table.

Expand table

Attribute	Description	Required or optional
`strength`	The relative duration of a pause by using one of the following values: • x-weak • weak • medium (default) • strong • x-strong	Optional
`time`	The absolute duration of a pause in seconds (such as `2s`) or milliseconds (such as `500ms`). Valid values range from 0 to 20000 milliseconds. If you set a value greater than the supported maximum, the service uses `20000ms`. If the `time` attribute is set, the `strength` attribute is ignored.	Optional

Here are more details about the `strength` attribute.

Expand table

Strength	Relative duration
X-weak	250 ms
Weak	500 ms
Medium	750 ms
Strong	1,000 ms
X-strong	1,250 ms

Break examples

The supported values for attributes of the `break` element were described previously. The following three ways all add 750 ms breaks.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        Welcome <break /> to text to speech.
        Welcome <break strength="medium" /> to text to speech.
        Welcome <break time="750ms" /> to text to speech.
    </voice>
</speak>
```

Add silence

Use the `mstts:silence` element to insert pauses before or after text, or between two adjacent sentences.

One of the differences between `mstts:silence` and `break` is that a `break` element can be inserted anywhere in the text. Silence only works at the beginning or end of input text or at the boundary of two adjacent sentences.

The silence setting is applied to all input text within its enclosing `voice` element. To reset or change the silence setting again, you must use a new `voice` element with either the same voice or a different voice.

Usage of the `mstts:silence` element's attributes are described in the following table.

Expand table

Attribute	Description	Required or optional
`type`	Specifies where and how to add silence. The following silence types are supported: • `Leading` – Extra silence at the beginning of the text. The value that you set is added to the natural silence before the start of text. • `Leading-exact` – Silence at the beginning of the text. The value is an absolute silence length. • `Tailing` – Extra silence at the end of text. The value that you set is added to the natural silence after the last word. • `Tailing-exact` – Silence at the end of the text. The value is an absolute silence length.	Required

Attribute	Description	Required or optional
	• `Sentenceboundary` – Extra silence between adjacent sentences. The actual silence length for this type includes the natural silence after the last word in the previous sentence, the value you set for this type, and the natural silence before the starting word in the next sentence. • `Sentenceboundary-exact` – Silence between adjacent sentences. The value is an absolute silence length. • `Comma-exact` – Silence at the comma in half-width or full-width format. The value is an absolute silence length. • `Semicolon-exact` – Silence at the semicolon in half-width or full-width format. The value is an absolute silence length. • `Enumerationcomma-exact` – Silence at the enumeration comma in full-width format. The value is an absolute silence length. An absolute silence type (with the `-exact` suffix) replaces any otherwise natural leading or trailing silence. Absolute silence types take precedence over the corresponding non-absolute type. For example, if you set both `Leading` and `Leading-exact` types, the `Leading-exact` type takes effect. The WordBoundary event takes precedence over punctuation-related silence settings including `Comma-exact`, `Semicolon-exact`, or `Enumerationcomma-exact`. When you use both the `WordBoundary` event and punctuation-related silence settings, the punctuation-related silence settings don't take effect.	
`Value`	The duration of a pause in seconds (such as `2s`) or milliseconds (such as `500ms`). Valid values range from 0 to 20000 milliseconds. If you set a value greater than the supported maximum, the service uses `20000ms`.	Required

mstts silence examples

The supported values for attributes of the `mstts:silence` element were described previously.

In this example, `mstts:silence` is used to add 200 ms of silence between two sentences.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" xml:lang="en-US">
<voice name="en-US-AvaNeural">
<mstts:silence  type="Sentenceboundary" value="200ms"/>
If we're home schooling, the best we can do is roll with what each day brings and
try to have fun along the way.
A good place to start is by trying out the slew of educational apps that are
helping children stay happy and smash their schooling at the same time.
```

```
</voice>
</speak>
```

In this example, `mstts:silence` is used to add 50 ms of silence at the comma, 100 ms of silence at the semicolon, and 150 ms of silence at the enumeration comma.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" xml:lang="zh-CN">
<voice name="zh-CN-YunxiNeural">
<mstts:silence type="comma-exact" value="50ms"/><mstts:silence type="semicolon-
exact" value="100ms"/><mstts:silence type="enumerationcomma-exact" value="150ms"/>
你好呀，云希、晓晓；你好呀。
</voice>
</speak>
```

Specify paragraphs and sentences

The `p` and `s` elements are used to denote paragraphs and sentences, respectively. In the absence of these elements, the Speech service automatically determines the structure of the SSML document.

Paragraph and sentence examples

The following example defines two paragraphs that each contain sentences. In the second paragraph, the Speech service automatically determines the sentence structure, since they aren't defined in the SSML document.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        <p>
            <s>Introducing the sentence element.</s>
            <s>Used to mark individual sentences.</s>
        </p>
        <p>
            Another simple paragraph.
            Sentence structure in this paragraph is not explicitly marked.
        </p>
    </voice>
</speak>
```

Bookmark element

You can use the `bookmark` element in SSML to reference a specific location in the text or tag sequence. Then you use the Speech SDK and subscribe to the `BookmarkReached` event to get the offset of each marker in the audio stream. The `bookmark` element isn't spoken. For more information, see Subscribe to synthesizer events.

Usage of the `bookmark` element's attributes are described in the following table.

Expand table

Attribute	Description	Required or optional
`mark`	The reference text of the `bookmark` element.	Required

Bookmark examples

The supported values for attributes of the `bookmark` element were described previously.

As an example, you might want to know the time offset of each flower word in the following snippet:

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        We are selling <bookmark mark='flower_1'/>roses and <bookmark
mark='flower_2'/>daisies.
    </voice>
</speak>
```

Next steps

- SSML overview
- Voice and sound with SSML
- Language support: Voices, locales, languages

Customize voice and sound with SSML

07/09/2025

You can use Speech Synthesis Markup Language (SSML) to specify the text to speech voice, language, name, style, and role for your speech output. You can also use multiple voices in a single SSML document, and adjust the emphasis, speaking rate, pitch, and volume. In addition, SSML features the ability to insert prerecorded audio, such as a sound effect or a musical note.

The article shows you how to use SSML elements to specify voice and sound. For more information about SSML syntax, see SSML document structure and events.

Use voice elements

At least one `voice` element must be specified within each SSML speak element. This element determines the voice that's used for text to speech.

You can include multiple `voice` elements in a single SSML document. Each `voice` element can specify a different voice. You can also use the same voice multiple times with different settings, such as when you change the silence duration between sentences.

The following table describes the usage of the `voice` element's attributes:

Expand table

Attribute	Description	Required or optional
`name`	The voice used for text to speech output. For a complete list of supported standard voices, see Language support.	Required
`effect`	The audio effect processor that's used to optimize the quality of the synthesized speech output for specific scenarios on devices. For some scenarios in production environments, the auditory experience might be degraded due to the playback distortion on certain devices. For example, the synthesized speech from a car speaker might sound dull and muffled due to environmental factors such as speaker response, room reverberation, and background noise. The passenger might have to turn up the volume to hear more clearly. To avoid manual operations in such a scenario, the audio effect processor can make the sound clearer by compensating the distortion of playback. The following values are supported:	Optional

Attribute	Description	Required or optional
	• `eq_car` – Optimize the auditory experience when providing high-fidelity speech in cars, buses, and other enclosed automobiles. • `eq_telecomhp8k` – Optimize the auditory experience for narrowband speech in telecom or telephone scenarios. You should use a sampling rate of 8 kHz. If the sample rate isn't 8 kHz, the auditory quality of the output speech isn't optimized. If the value is missing or invalid, this attribute is ignored and no effect is applied.	

Voice examples

For information about the supported values for attributes of the `voice` element, see Use voice elements.

Single voice example

This example uses the `en-US-AvaMultilingualNeural` voice.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        This is the text that is spoken.
    </voice>
</speak>
```

Multiple voices example

Within the `speak` element, you can specify multiple voices for text to speech output. These voices can be in different languages. For each voice, the text must be wrapped in a `voice` element.

This example alternates between the `en-US-AvaMultilingualNeural` and `en-US-AndrewMultilingualNeural` voices. The neural multilingual voices can speak different languages based on the input text.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        Good morning!
    </voice>
    <voice name="en-US-AndrewMultilingualNeural">
        Good morning to you too Ava!
    </voice>
</speak>
```

Custom voice example

To use your custom voice, specify the model name as the voice name in SSML.

This example uses a custom voice named **my-custom-voice**.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="my-custom-voice">
        This is the text that is spoken.
    </voice>
</speak>
```

Audio effect example

You use the `effect` attribute to optimize the auditory experience for scenarios such as cars and telecommunications. The following SSML example uses the `effect` attribute with the configuration in car scenarios.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural" effect="eq_car">
        This is the text that is spoken.
    </voice>
</speak>
```

Multi-talker voice example

Multi-talker voices enable natural, dynamic conversations with multiple distinct speakers. This innovation enhances the realism of synthesized dialogues by preserving contextual flow, emotional consistency, and natural speech patterns.

Use this capability to generate engaging, podcast-style speech or conversational exchanges with seamless transitions between speakers. Unlike single-talker models, which synthesize each turn in isolation, multi-talker voices maintain coherence across dialogue, ensuring a more authentic and immersive listening experience.

For `en-US-MultiTalker-Ava-Andrew:DragonHDLatestNeural`, within the `<mstts:dialog>` element, you can specify each turn for the text to speech output, with the format below to alternates between the speaker `ava` and `andrew` for each turn.

XML

```
<speak version='1.0' xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='https://www.w3.org/2001/mstts' xml:lang='en-US'>
    <voice name='en-US-MultiTalker-Ava-Andrew:DragonHDLatestNeural'>
        <mstts:dialog>
            <mstts:turn speaker="ava">Hello, Andrew! How's your day going?
</mstts:turn>
            <mstts:turn speaker="andrew">Hey Ava! It's been great, just exploring
some AI advancements in communication.</mstts:turn>
            <mstts:turn speaker="ava">That sounds interesting! What kind of
projects are you working on?</mstts:turn>
            <mstts:turn speaker="andrew">Well, we've been experimenting with text-
to-speech applications, including turning emails into podcasts.</mstts:turn>
            <mstts:turn speaker="ava">Wow, that could really improve content
accessibility! Are you looking for collaborators?</mstts:turn>
            <mstts:turn speaker="andrew">Absolutely! We're open to testing new
ideas and seeing how AI can enhance communication.</mstts:turn>
        </mstts:dialog>
    </voice>
</speak>
```

For supported voices, see the Language support documentation.

Use speaking styles and roles

By default, neural voices have a neutral speaking style. You can adjust the speaking style, style degree, and role at the sentence level.

> **Note**
>
> The Speech service supports styles, style degree, and roles for a subset of neural voices as described in the voice styles and roles documentation. To determine the supported styles and roles for each voice, you can also use the list voices API and the audio content creation web application.

The following table describes the usage of the `mstts:express-as` element's attributes:

Expand table

Attribute	Description	Required or optional
`style`	The voice-specific speaking style. You can express emotions like cheerfulness, empathy, and calmness. You can also optimize the voice for different scenarios like customer service, newscast, and voice assistant. If the style value is missing or invalid, the entire `mstts:express-as` element is ignored and the service uses the default neutral speech. For custom voice styles, see the custom voice style example.	Required
`styledegree`	The intensity of the speaking style. You can specify a stronger or softer style to make the speech more expressive or subdued. The range of accepted values are: `0.01` to `2` inclusive. The default value is `1`, which means the predefined style intensity. The minimum unit is `0.01`, which results in a slight tendency for the target style. A value of `2` results in a doubling of the default style intensity. If the style degree is missing or isn't supported for your voice, this attribute is ignored.	Optional
`role`	The speaking role-play. The voice can imitate a different age and gender, but the voice name isn't changed. For example, a male voice can raise the pitch and change the intonation to imitate a female voice, but the voice name isn't changed. If the role is missing or isn't supported for your voice, this attribute is ignored.	Optional

The following table describes each supported `style` attribute:

Expand table

Style	Description
`style="advertisement_upbeat"`	Expresses an excited and high-energy tone for promoting a product or service.
`style="affectionate"`	Expresses a warm and affectionate tone, with higher pitch and vocal energy. The speaker is in a state of attracting the attention of the listener. The personality of the speaker is often endearing in nature.
`style="angry"`	Expresses an angry and annoyed tone.
`style="assistant"`	Expresses a warm and relaxed tone for digital assistants.
`style="calm"`	Expresses a cool, collected, and composed attitude when speaking. Tone, pitch, and prosody are more uniform compared to other

Style	Description
	types of speech.
`style="chat"`	Expresses a casual and relaxed tone.
`style="cheerful"`	Expresses a positive and happy tone.
`style="customerservice"`	Expresses a friendly and helpful tone for customer support.
`style="depressed"`	Expresses a melancholic and despondent tone with lower pitch and energy.
`style="disgruntled"`	Expresses a disdainful and complaining tone. Speech of this emotion displays displeasure and contempt.
`style="documentary-narration"`	Narrates documentaries in a relaxed, interested, and informative style suitable for documentaries, expert commentary, and similar content.
`style="embarrassed"`	Expresses an uncertain and hesitant tone when the speaker is feeling uncomfortable.
`style="empathetic"`	Expresses a sense of caring and understanding.
`style="envious"`	Expresses a tone of admiration when you desire something that someone else has.
`style="excited"`	Expresses an upbeat and hopeful tone. It sounds like something great is happening and the speaker is happy about it.
`style="fearful"`	Expresses a scared and nervous tone, with higher pitch, higher vocal energy, and faster rate. The speaker is in a state of tension and unease.
`style="friendly"`	Expresses a pleasant, inviting, and warm tone. It sounds sincere and caring.
`style="gentle"`	Expresses a mild, polite, and pleasant tone, with lower pitch and vocal energy.
`style="hopeful"`	Expresses a warm and yearning tone. It sounds like something good is expected to happen to the speaker.
`style="lyrical"`	Expresses emotions in a melodic and sentimental way.
`style="narration-professional"`	Expresses a professional, objective tone for content reading.
`style="narration-relaxed"`	Expresses a soothing and melodious tone for content reading.
`style="newscast"`	Expresses a formal and professional tone for narrating news.
`style="newscast-casual"`	Expresses a versatile and casual tone for general news delivery.

Style	Description
`style="newscast-formal"`	Expresses a formal, confident, and authoritative tone for news delivery.
`style="poetry-reading"`	Expresses an emotional and rhythmic tone while reading a poem.
`style="sad"`	Expresses a sorrowful tone.
`style="serious"`	Expresses a strict and commanding tone. Speaker often sounds stiffer and much less relaxed with firm cadence.
`style="shouting"`	Expresses a tone that sounds as if the voice is distant or in another location and making an effort to be clearly heard.
`style="sports_commentary"`	Expresses a relaxed and interested tone for broadcasting a sports event.
`style="sports_commentary_excited"`	Expresses an intensive and energetic tone for broadcasting exciting moments in a sports event.
`style="whispering"`	Expresses a soft tone that's trying to make a quiet and gentle sound.
`style="terrified"`	Expresses a scared tone, with a faster pace and a shakier voice. It sounds like the speaker is in an unsteady and frantic status.
`style="unfriendly"`	Expresses a cold and indifferent tone.

The following table has descriptions of each supported `role` attribute:

Expand table

Role	Description
`role="Girl"`	The voice imitates a girl.
`role="Boy"`	The voice imitates a boy.
`role="YoungAdultFemale"`	The voice imitates a young adult female.
`role="YoungAdultMale"`	The voice imitates a young adult male.
`role="OlderAdultFemale"`	The voice imitates an older adult female.
`role="OlderAdultMale"`	The voice imitates an older adult male.
`role="SeniorFemale"`	The voice imitates a senior female.
`role="SeniorMale"`	The voice imitates a senior male.

mstts express-as examples

For information about the supported values for attributes of the `mstts:express-as` element, see Use speaking styles and roles.

Style and degree example

You use the `mstts:express-as` element to express emotions like cheerfulness, empathy, and calm. You can also optimize the voice for different scenarios like customer service, newscast, and voice assistant.

The following SSML example uses the `<mstts:express-as>` element with a `sad` style degree of `2`.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="zh-CN">
    <voice name="zh-CN-XiaomoNeural">
        <mstts:express-as style="sad" styledegree="2">
            快走吧，路上一定要注意安全，早去早回。
        </mstts:express-as>
    </voice>
</speak>
```

Role example

Apart from adjusting the speaking styles and style degree, you can also adjust the `role` parameter so that the voice imitates a different age and gender. For example, a male voice can raise the pitch and change the intonation to imitate a female voice, but the voice name isn't changed.

This SSML snippet illustrates how the `role` attribute is used to change the role-play for `zh-CN-XiaomoNeural`.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="zh-CN">
    <voice name="zh-CN-XiaomoNeural">
        女儿看见父亲走了进来，问道：
        <mstts:express-as role="YoungAdultFemale" style="calm">
            “您来的挺快的，怎么过来的？”
        </mstts:express-as>
        父亲放下手提包，说：
```

```
        <mstts:express-as role="OlderAdultMale" style="calm">
            “刚打车过来的，路上还挺顺畅。”
        </mstts:express-as>
    </voice>
</speak>
```

Custom voice style example

You can train your custom voice to speak with some preset styles such as `cheerful`, `sad`, and `whispering`. You can also fine-tune a professional voice to speak in a custom style as determined by your training data. To use your custom voice style in SSML, specify the style name that you previously entered in Speech Studio.

This example uses a custom voice named **my-custom-voice**. The custom voice speaks with the `cheerful` preset style and style degree of `2`, and then with a custom style named **my-custom-style** and style degree of `0.01`.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
    <voice name="my-custom-voice">
        <mstts:express-as style="cheerful" styledegree="2">
            That'd be just amazing!
        </mstts:express-as>
        <mstts:express-as style="my-custom-style" styledegree="0.01">
            What's next?
        </mstts:express-as>
    </voice>
</speak>
```

Speaker profile ID

You use the `mstts:ttsembedding` element to specify the `speakerProfileId` property for a personal voice. Personal voice is a custom voice trained on your own voice or your customer's voice. For more information, see create a personal voice.

The following SSML example uses the `<mstts:ttsembedding>` element with a voice name and speaker profile ID.

XML

```
<speak version='1.0' xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='http://www.w3.org/2001/mstts' xml:lang='en-US'>
    <voice xml:lang='en-US' xml:gender='Male' name='PhoenixV2Neural'>
```

```
    <mstts:ttsembedding speakerProfileId='your speaker profile ID here'>
    I'm happy to hear that you find me amazing and that I have made your trip
planning easier and more fun. 我很高兴听到你觉得我很了不起，我让你的旅行计划更轻松、更有趣。Je suis heureux d'apprendre que vous me trouvez incroyable et que j'ai rendu la planification de votre voyage plus facile et plus amusante.
    </mstts:ttsembedding>
    </voice>
</speak>
```

Adjust speaking languages

By default, multilingual voices can autodetect the language of the input text and speak in the language of the default locale of the input text without using SSML. Optionally, you can use the `<lang xml:lang>` element to adjust the speaking language for these voices to set the preferred accent such as `en-GB` for British English. You can adjust the speaking language at both the sentence level and word level. For information about the supported languages for multilingual voice, see Multilingual voices with the lang element for a table showing the `<lang>` syntax and attribute definitions.

The following table describes the usage of the `<lang xml:lang>` element's attributes:

Expand table

Attribute	Description	Required or optional
`xml:lang`	The language that you want the neural voice to speak.	Required to adjust the speaking language for the neural voice. If you're using `lang xml:lang`, the locale must be provided.

> **Note**
>
> The `<lang xml:lang>` element is incompatible with the `prosody` and `break` elements. You can't adjust pause and prosody like pitch, contour, rate, or volume in this element.
>
> Non-multilingual voices don't support the `<lang xml:lang>` element by design.

Multilingual voices with the lang element

Use the multilingual voices section to determine which speaking languages the Speech service supports for each neural voice, as demonstrated in the following example table. If the voice doesn't speak the language of the input text, the Speech service doesn't output synthesized audio.

Expand table

Voice	Auto-detected language number	Auto-detected language (locale)	All locales number	All languages (locale) supported from SSML
`en-US-AndrewMultilingualNeural`[1] (Male) `en-US-AvaMultilingualNeural`[1] (Female) `en-US-BrianMultilingualNeural`[1] (Male) `en-US-EmmaMultilingualNeural`[1] (Female)	77	Afrikaans (`af-ZA`), Albanian (`sq-AL`), Amharic (`am-ET`), Arabic (`ar-EG`), Armenian (`hy-AM`), Azerbaijani (`az-AZ`), Bahasa Indonesian (`id-ID`), Bangla (`bn-BD`), Basque (`eu-ES`), Bengali (`bn-IN`), Bosnian (`bs-BA`), Bulgarian (`bg-BG`), Burmese (`my-MM`), Catalan (`ca-ES`), Chinese Cantonese (`zh-HK`), Chinese Mandarin (`zh-CN`), Chinese Taiwanese (`zh-TW`), Croatian (`hr-HR`), Czech (`cs-CZ`), Danish (`da-DK`), Dutch (`nl-NL`), English (`en-US`), Estonian (`et-EE`), Filipino (`fil-PH`), Finnish (`fi-FI`), French (`fr-FR`), Galician (`gl-ES`), Georgian (`ka-GE`), German (`de-DE`), Greek (`el-GR`), Hebrew (`he-IL`), Hindi (`hi-IN`), Hungarian (`hu-HU`), Icelandic (`is-IS`), Irish (`ga-IE`), Italian (`it-IT`), Japanese (`ja-JP`), Javanese (`jv-ID`), Kannada (`kn-IN`), Kazakh (`kk-KZ`), Khmer (`km-KH`),	91	Afrikaans (South Africa) (`af-ZA`), Albanian (Albania) (`sq-AL`), Amharic (Ethiopia) (`am-ET`), Arabic (Egypt) (`ar-EG`), Arabic (Saudi Arabia) (`ar-SA`), Armenian (Armenia) (`hy-AM`), Azerbaijani (Azerbaijan) (`az-AZ`), Basque (Basque) (`eu-ES`), Bengali (India) (`bn-IN`), Bosnian (Bosnia and Herzegovina) (`bs-BA`), Bulgarian (Bulgaria) (`bg-BG`), Burmese (Myanmar) (`my-MM`), Catalan (Spain) (`ca-ES`), Chinese (Cantonese, Traditional) (`zh-HK`), Chinese (Mandarin, Simplified) (`zh-CN`), Chinese (Taiwanese Mandarin) (`zh-TW`), Croatian (Croatia) (`hr-HR`), Czech (Czech) (`cs-CZ`), Danish (Denmark) (`da-DK`), Dutch (Belgium) (`nl-BE`), Dutch (Netherlands) (`nl-NL`), English (Australia) (`en-AU`), English (Canada) (`en-CA`), English (Hong Kong SAR) (`en-HK`), English (India) (`en-IN`), English (Ireland) (`en-IE`), English (United Kingdom) (`en-GB`), English (United States) (`en-US`), Estonian (Estonia) (`et-EE`), Filipino (Philippines) (`fil-PH`), Finnish (Finland) (`fi-FI`), French (Belgium) (`fr-BE`), French (Canada) (`fr-CA`), French (France) (`fr-FR`), French (Switzerland) (`fr-CH`), Galician (Galician) (`gl-ES`), Georgian (Georgia) (`ka-GE`),

Voice	Auto-detected language number	Auto-detected language (locale)	All locales number	All languages (locale) supported from SSML
		Korean (ko-KR), Lao (lo-LA), Latvian (lv-LV), Lithuanian (lt-LT), Macedonian (mk-MK), Malay (ms-MY), Malayalam (ml-IN), Maltese (mt-MT), Mongolian (mn-MN), Nepali (ne-NP), Norwegian Bokmål (nb-NO), Pashto (ps-AF), Persian (fa-IR), Polish (pl-PL), Portuguese (pt-BR), Romanian (ro-RO), Russian (ru-RU), Serbian (sr-RS), Sinhala (si-LK), Slovak (sk-SK), Slovene (sl-SI), Somali (so-SO), Spanish (es-ES), Sundanese (su-ID), Swahili (sw-KE), Swedish (sv-SE), Tamil (ta-IN), Telugu (te-IN), Thai (th-TH), Turkish (tr-TR), Ukrainian (uk-UA), Urdu (ur-PK), Uzbek (uz-UZ), Vietnamese (vi-VN), Welsh (cy-GB), Zulu (zu-ZA)		German (Austria) (de-AT), German (Germany) (de-DE), German (Switzerland) (de-CH), Greek (Greece) (el-GR), Hebrew (Israel) (he-IL), Hindi (India) (hi-IN), Hungarian (Hungary) (hu-HU), Icelandic (Iceland) (is-IS), Indonesian (Indonesia) (id-ID), Irish (Ireland) (ga-IE), Italian (Italy) (it-IT), Japanese (Japan) (ja-JP), Javanese (Indonesia) (jv-ID), Kannada (India) (kn-IN), Kazakh (Kazakhstan) (kk-KZ), Khmer (Cambodia) (km-KH), Korean (Korea) (ko-KR), Lao (Laos) (lo-LA), Latvian (Latvia) (lv-LV), Lithuanian (Lithuania) (lt-LT), Macedonian (North Macedonia) (mk-MK), Malay (Malaysia) (ms-MY), Malayalam (India) (ml-IN), Maltese (Malta) (mt-MT), Mongolian (Mongolia) (mn-MN), Nepali (Nepal) (ne-NP), Norwegian (Bokmål, Norway) (nb-NO), Pashto (Afghanistan) (ps-AF), Persian (Iran) (fa-IR), Polish (Poland) (pl-PL), Portuguese (Brazil) (pt-BR), Portuguese (Portugal) (pt-PT), Romanian (Romania) (ro-RO), Russian (Russia) (ru-RU), Serbian (Cyrillic, Serbia) (sr-RS), Sinhala (Sri Lanka) (si-LK), Slovak (Slovakia) (sk-SK), Slovenian (Slovenia) (sl-SI), Somali (Somalia) (so-SO), Spanish (Mexico) (es-MX), Spanish (Spain) (es-ES), Sundanese (Indonesia) (su-ID), Swahili (Kenya) (sw-

Voice	Auto-detected language number	Auto-detected language (locale)	All locales number	All languages (locale) supported from SSML
				KE), Swedish (Sweden) (sv-SE), Tamil (India) (ta-IN), Telugu (India) (te-IN), Thai (Thailand) (th-TH), Turkish (Türkiye) (tr-TR), Ukrainian (Ukraine) (uk-UA), Urdu (Pakistan) (ur-PK), Uzbek (Uzbekistan) (uz-UZ), Vietnamese (Vietnam) (vi-VN), Welsh (United Kingdom) (cy-GB), Zulu (South Africa) (zu-ZA)

[1] Those are neural multilingual voices in Azure AI Speech. All multilingual voices can speak in the language in default locale of the input text without using SSML. However, you can still use the `<lang xml:lang>` element to adjust the speaking accent of each language to set preferred accent such as British accent (`en-GB`) for English. The prefix in each voice name indicates its primary locale; for example, the primary locale for `en-US-AndrewMultilingualNeural` is `en-US`.

> **Note**
>
> Multilingual voices don't fully support certain SSML elements, such as `break`, `emphasis`, `silence`, and `sub`.

Lang examples

For information about the supported values for attributes of the `lang` element, see Adjust speaking language.

You must specify `en-US` as the default language within the `speak` element, whether or not the language is adjusted elsewhere. In this example, the primary language for `en-US-AvaMultilingualNeural` is `en-US`.

This SSML snippet shows how to use `<lang xml:lang>` to speak `de-DE` with the `en-US-AvaMultilingualNeural` neural voice.

```
XML
```

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        <lang xml:lang="de-DE">
            Wir freuen uns auf die Zusammenarbeit mit Ihnen!
        </lang>
    </voice>
</speak>
```

Within the `speak` element, you can specify multiple languages including `en-US` for text to speech output. For each adjusted language, the text must match the language and be wrapped in a `voice` element. This SSML snippet shows how to use `<lang xml:lang>` to change the speaking languages to `es-MX`, `en-US`, and `fr-FR`.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        <lang xml:lang="es-MX">
            ¡Esperamos trabajar con usted!
        </lang>
        <lang xml:lang="en-US">
           We look forward to working with you!
        </lang>
        <lang xml:lang="fr-FR">
            Nous avons hâte de travailler avec vous!
        </lang>
    </voice>
</speak>
```

Adjust prosody

You can use the `prosody` element to specify changes to pitch, contour, range, rate, and volume for the text to speech output. The `prosody` element can contain text and the following elements: `audio`, `break`, `p`, `phoneme`, `prosody`, `say-as`, `sub`, and `s`.

Because prosodic attribute values can vary over a wide range, the speech recognizer interprets the assigned values as a suggestion of what the actual prosodic values of the selected voice should be. Text to speech limits or substitutes values that aren't supported. Examples of unsupported values are a pitch of 1 MHz or a volume of 120.

The following table describes the usage of the `prosody` element's attributes:

Expand table

Attribute	Description	Required or optional
`contour`	Contour represents changes in pitch. These changes are represented as an array of targets at specified time positions in the speech output. Sets of parameter pairs define each target. For example: `<prosody contour="(0%,+20Hz) (10%,-2st) (40%,+10Hz)">` The first value in each set of parameters specifies the location of the pitch change as a percentage of the duration of the text. The second value specifies the amount to raise or lower the pitch by using a relative value or an enumeration value for pitch (see `pitch`). Pitch contour doesn't work on single words and short phrases. It's recommended to adjust the pitch contour on whole sentences or long phrases.	Optional

`pitch`	Indicates the baseline pitch for the text. Pitch changes can be applied at the sentence level. The pitch changes should be within 0.5 to 1.5 times the original audio. You can express the pitch as:
	• An absolute value: Expressed as a number followed by "Hz" (Hertz). For example, `<prosody pitch="600Hz">some text</prosody>`.
	• A relative value:
	  ◦ As a relative number: Expressed as a number preceded by "+" or "-" and followed by "Hz" or "st" that specifies an amount to change the pitch. For example: `<prosody pitch="+80Hz">some text</prosody>` or `<prosody pitch="-2st">some text</prosody>`. The "st" indicates the change unit is semitone, which is half of a tone (a half step) on the standard diatonic scale.
	  ◦ As a percentage: Expressed as a number preceded by "+" (optionally) or "-" and followed by "%", indicating the relative change. For example: `<prosody pitch="50%">some text</prosody>` or `<prosody pitch="-50%">some text</prosody>`.
	• A constant value:
	  ◦ `x-low` (equivalently 0.55,-45%)
	  ◦ `low` (equivalently 0.8, -20%)
	  ◦ `medium` (equivalently 1, default value)
	  ◦ `high` (equivalently 1.2, +20%)
	  ◦ `x-high` (equivalently 1.45, +45%)
		Optional

`range`	A value that represents the range of pitch for the text. You can express `range` by using the same absolute values, relative values, or enumeration values used to describe `pitch`.	Optional
`rate`	Indicates the speaking rate of the text. Speaking rate can be applied at the word or sentence level. The rate changes should be within `0.5` to `2` times the original audio. You can express `rate` as:	Optional

Attribute	Description	Required or optional

	• A relative value:
	  ◦ As a relative number: Expressed as a number that acts as a multiplier of the default. For example, a value of `1` results in no change in the original rate. A value of `0.5` results in a halving of the original rate. A value of `2` results in twice the original rate.
	  ◦ As a percentage: Expressed as a number preceded by "+" (optionally) or "-" and followed by "%", indicating the relative change. For example: `<prosody rate="50%">some text</prosody>` or `<prosody rate="-50%">some text</prosody>`.
	• A constant value:
	  ◦ `x-slow` (equivalently 0.5, -50%)
	  ◦ `slow` (equivalently 0.64, -46%)
	  ◦ `medium` (equivalently 1, default value)
	  ◦ `fast` (equivalently 1.55, +55%)
	  ◦ `x-fast` (equivalently 2, +100%)	

`volume`	Indicates the volume level of the speaking voice. Volume changes can be applied at the sentence level. You can express the volume as:
	• An absolute value: Expressed as a number in the range of `0.0` to `100.0`, from *quietest* to *loudest*, such as `75`. The default value is `100.0`.
	• A relative value:
	  ◦ As a relative number: Expressed as a number preceded by "+" or "-" that specifies an amount to change the volume. Examples are `+10` or `-5.5`.
	  ◦ As a percentage: Expressed as a number preceded by "+" (optionally) or "-" and followed by "%", indicating the relative change. For example: `<prosody volume="50%">some text</prosody>` or `<prosody volume="+3%">some text</prosody>`.
	• A constant value:
	  ◦ `silent` (equivalently 0)
	  ◦ `x-soft` (equivalently 0.2)
	  ◦ `soft` (equivalently 0.4)
	  ◦ `medium` (equivalently 0.6)
	  ◦ `loud` (equivalently 0.8)
	  ◦ `x-loud` (equivalently 1, default value)
		Optional

Prosody examples

For information about the supported values for attributes of the `prosody` element, see Adjust prosody.

Change speaking rate example

This SSML snippet illustrates how the `rate` attribute is used to change the speaking rate to 30% greater than the default rate.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        <prosody rate="+30.00%">
            Enjoy using text to speech.
        </prosody>
    </voice>
</speak>
```

Change volume example

This SSML snippet illustrates how the `volume` attribute is used to change the volume to 20% greater than the default volume.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        <prosody volume="+20.00%">
            Enjoy using text to speech.
        </prosody>
    </voice>
</speak>
```

Change pitch example

This SSML snippet illustrates how the `pitch` attribute is used so that the voice speaks in a high pitch.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        Welcome to <prosody pitch="high">Enjoy using text to speech.</prosody>
    </voice>
</speak>
```

Change pitch contour example

This SSML snippet illustrates how the `contour` attribute is used to change the contour.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        <prosody contour="(60%,-60%) (100%,+80%)" >
            Were you the only person in the room?
        </prosody>
    </voice>
</speak>
```

Adjust emphasis

You can use the optional `emphasis` element to add or remove word-level stress for the text. This element can only contain text and the following elements: `audio`, `break`, `emphasis`, `lang`, `phoneme`, `prosody`, `say-as`, `sub`, and `voice`.

> **Note**
>
> The word-level emphasis tuning is only available for these neural voices: `en-US-GuyNeural`, `en-US-DavisNeural`, and `en-US-JaneNeural`.
>
> For words that have low pitch and short duration, the pitch might not be raised enough to be noticed.

The following table describes the `emphasis` element's attributes:

Expand table

Attribute	Description	Required or optional
`level`	Indicates the strength of emphasis to be applied: • `reduced` • `none` • `moderate` • `strong` When the `level` attribute isn't specified, the default level is `moderate`. For details on each attribute, see emphasis element.	Optional

Emphasis examples

For information about the supported values for attributes of the `emphasis` element, see Adjust emphasis.

This SSML snippet demonstrates how you can use the `emphasis` element to add moderate level emphasis for the word "meetings."

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
    <voice name="en-US-AndrewMultilingualNeural">
    I can help you join your <emphasis level="moderate">meetings</emphasis> fast.
    </voice>
</speak>
```

Add recorded audio

The `audio` element is optional. You can use it to insert prerecorded audio into an SSML document. The body of the `audio` element can contain plain text or SSML markup spoken if the audio file is unavailable or unplayable. The `audio` element can also contain text and the following elements: `audio`, `break`, `p`, `s`, `phoneme`, `prosody`, `say-as`, and `sub`.

Any audio included in the SSML document must meet these requirements:

- The audio file must be valid *.*mp3*, *.*wav*, *.*opus*, *.*ogg*, *.*flac*, or *.*wma* files.
- The combined total time for all text and audio files in a single response can't exceed 600 seconds.
- The audio must not contain any customer-specific or other sensitive information.

> **Note**
>
> The `audio` element isn't supported by the Long Audio API. For long-form text to speech, use the batch synthesis API instead.

The following table describes the usage of the `audio` element's attributes:

Expand table

Attribute	Description	Required or optional
`src`	The URI location of the audio file. The audio must be hosted on an internet-accessible HTTPS endpoint. HTTPS is required. The domain hosting the file must present a valid, trusted TLS/SSL certificate. You should put the audio file into Blob Storage in the same Azure region as the text to speech endpoint to minimize the latency.	Required

Audio examples

For information about the supported values for attributes of the `audio` element, see Add recorded audio.

This SSML snippet illustrates how to use `src` attribute to insert audio from two .wav files.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        <p>
            <audio src="https://contoso.com/opinionprompt.wav"/>
            Thanks for offering your opinion. Please begin speaking after the
beep.
            <audio src="https://contoso.com/beep.wav">
                Could not play the beep, please voice your opinion now.
            </audio>
        </p>
    </voice>
</speak>
```

Adjust the audio duration

Use the `mstts:audioduration` element to set the duration of the output audio. Use this element to help synchronize the timing of audio output completion. The audio duration can be decreased or increased between `0.5` to `2` times the rate of the original audio. The original audio is the audio without any other rate settings. The speaking rate is slowed down or sped up accordingly based on the set value.

The audio duration setting applies to all input text within its enclosing `voice` element. To reset or change the audio duration setting again, you must use a new `voice` element with either the same voice or a different voice.

The following table describes the usage of the `mstts:audioduration` element's attributes:

Expand table

Attribute	Description	Required or optional
`value`	The requested duration of the output audio in either seconds, such as `2s`, or milliseconds, such as `2000ms`. The maximum value for output audio duration is 300 seconds. This value should be within `0.5` to `2` times the original audio without any other rate settings. For example, if the requested duration of your audio is `30s`, then the original audio must otherwise be between 15 and 60 seconds. If you set a value outside of these boundaries, the duration is set according to the respective minimum or maximum multiple. For output audio longer than 300 seconds, first generate the original audio without any other rate settings, then calculate the rate to adjust using the prosody rate to achieve the desired duration.	Required

mstts audio duration examples

For information about the supported values for attributes of the `mstts:audioduration` element, see Adjust the audio duration.

In this example, the original audio is around 15 seconds. The `mstts:audioduration` element is used to set the audio duration to 20 seconds or `20s`.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" xml:lang="en-US">
<voice name="en-US-AvaMultilingualNeural">
<mstts:audioduration value="20s"/>
If we're home schooling, the best we can do is roll with what each day brings and
try to have fun along the way.
A good place to start is by trying out the slew of educational apps that are
helping children stay happy and smash their schooling at the same time.
</voice>
</speak>
```

Add background audio

You can use the `mstts:backgroundaudio` element to add background audio to your SSML documents or mix an audio file with text to speech. With `mstts:backgroundaudio`, you can loop an audio file in the background, fade in at the beginning of text to speech, and fade out at the end of text to speech.

If the background audio provided is shorter than the text to speech or the fade out, it loops. If it's longer than the text to speech, it stops when the fade out is finished.

Only one background audio file is allowed per SSML document. You can intersperse `audio` tags within the `voice` element to add more audio to your SSML document.

> **Note**
>
> The `mstts:backgroundaudio` element should be put in front of all `voice` elements. If specified, it must be the first child of the `speak` element.
>
> The `mstts:backgroundaudio` element isn't supported by the Long Audio API. For long-form text to speech, use the batch synthesis API (Preview) instead.

The following table describes the usage of the `mstts:backgroundaudio` element's attributes:

Expand table

Attribute	Description	Required or optional
`src`	The URI location of the background audio file.	Required
`volume`	The volume of the background audio file. Accepted values: `0` to `100` inclusive. The default value is `1`.	Optional
`fadein`	The duration of the background audio fade-in as milliseconds. The default value is `0`, which is the equivalent to no fade in. Accepted values: `0` to `10000` inclusive.	Optional
`fadeout`	The duration of the background audio fade-out in milliseconds. The default value is `0`, which is the equivalent to no fade out. Accepted values: `0` to `10000` inclusive.	Optional

mstss backgroundaudio examples

For information about the supported values for attributes of the `mstts:backgroundaudi` element, see Add background audio.

XML

```xml
<speak version="1.0" xml:lang="en-US" xmlns:mstts="http://www.w3.org/2001/mstts">
    <mstts:backgroundaudio src="https://contoso.com/sample.wav" volume="0.7"
fadein="3000" fadeout="4000"/>
    <voice name="en-US-AvaMultilingualNeural">
```

```
        The text provided in this document are spoken over the background audio.
    </voice>
</speak>
```

Viseme element

A viseme is the visual description of a phoneme in spoken language. It defines the position of the face and mouth while a person is speaking. You can use the `mstts:viseme` element in SSML to request viseme output. For more information, see Get facial position with viseme.

The viseme setting is applied to all input text within its enclosing `voice` element. To reset or change the viseme setting again, you must use a new `voice` element with either the same voice or a different voice.

Usage of the `viseme` element's attributes are described in the following table.

Expand table

Attribute	Description	Required or optional
`type`	The type of viseme output. • `redlips_front` – lip-sync with viseme ID and audio offset output • `FacialExpression` – blend shapes output	Required

> **Note**
>
> Currently, `redlips_front` only supports neural voices in `en-US` locale, and `FacialExpression` supports neural voices in `en-US` and `zh-CN` locales.

Viseme examples

The supported values for attributes of the `viseme` element were described previously.

This SSML snippet illustrates how to request blend shapes with your synthesized speech.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="http://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="en-US-AvaNeural">
```

```
    <mstts:viseme type="FacialExpression"/>
    Rainbow has seven colors: Red, orange, yellow, green, blue, indigo, and
violet.
  </voice>
</speak>
```

Voice conversion element

Voice conversion (preview) is the process of transforming the voice characteristics of a given audio to a target voice speaker. After voice conversion, the resulting audio reserves source audio's linguistic content and prosody while the voice timbre sounds like the target speaker. For more information, see voice conversion.

Use the `<mstts:voiceconversion>` tag via Speech Synthesis Markup Language (SSML) to specify the source audio URL and the target voice for the conversion. For a complete list of supported target voices, see supported voices for voice conversion.

The following table describes the usage of the `mstts:voiceconversion` element's attributes:

Expand table

Attribute	Description	Required or optional
`url`	The URL of the source audio file that provides linguistic content and prosody for the synthesized speech. The `url` must be accessible via HTTPS URL. For example, `https://example.com/source.wav` Input audio must be under 100 MB.	Required

Here's how the voice conversion works:

- The source audio is a prerecorded audio file that contains the spoken words and prosody.
  - Text content: The final synthesized speech follows the spoken words in the source audio.
  - Prosody and rhythm: The speech maintains the timing and intonation from the source.
- The `<voice>` tag specifies the target voice used for the output audio. For information about the supported target voices, see supported voices for voice conversion.
- The output audio keeps the timbre (tone and voice quality) of the target voice, but follows the text and speaking style of the source audio.

> **Note**
>
> All SSML elements related to prosody and pronunciation such as `<prosody>` or `<mstts:express-as>` are ignored.
>
> Text input is optional and any text included in the SSML are ignored during rendering.

mstss voiceconversion examples

The following example demonstrates how to use `<mstts:voiceconversion>` to synthesize speech using a **target neural voice** while **matching both the content and prosody** of a given source audio:

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
    <voice xml:lang="en-US" xml:gender="Female" name="en-US-
AvaMultilingualNeural">
        <mstts:voiceconversion
url="https://your.blob.core.windows.net/sourceaudio.wav"/>
    </voice>
</speak>
```

Next steps

- SSML overview
- SSML document structure and events
- Language and voice support for the Speech service

Pronunciation with SSML

09/26/2025

You can use Speech Synthesis Markup Language (SSML) with text to speech to specify how the speech is pronounced. For example, you can use SSML with phonemes and a custom lexicon to improve pronunciation. You can also use SSML to define how a word or mathematical expression is pronounced.

Refer to the following sections for details about how to use SSML elements to improve pronunciation. For more information about SSML syntax, see SSML document structure and events.

phoneme element

The `phoneme` element is used for phonetic pronunciation in SSML documents. Always provide human-readable speech as a fallback.

Phonetic alphabets are composed of phones, which are made up of letters, numbers, or characters, sometimes in combination. Each phone describes a unique sound of speech. The phonetic alphabet is in contrast to the Latin alphabet, where any letter might represent multiple spoken sounds. Consider the different `en-US` pronunciations of the letter "c" in the words "candy" and "cease" or the different pronunciations of the letter combination "th" in the words "thing" and "those."

> **Note**
>
> For a list of locales that support phonemes, see footnotes in the language support table.

Usage of the `phoneme` element's attributes are described in the following table.

Expand table

Attribute	Description	Required or optional
`alphabet`	The phonetic alphabet to use when you synthesize the pronunciation of the string in the `ph` attribute. The string that specifies the alphabet must be specified in lowercase letters. The following options are the possible alphabets that you can specify: • `ipa` – See SSML phonetic alphabets • `sapi` – See SSML phonetic alphabets	Optional

Attribute	Description	Required or optional
	• `ups` – See Universal Phone Set • `x-sampa` – See SSML phonetic alphabets The alphabet applies only to the `phoneme` in the element.	
`ph`	A string containing phones that specify the pronunciation of the word in the `phoneme` element. If the specified string contains unrecognized phones, text to speech rejects the entire SSML document and produces none of the speech output specified in the document. For `ipa`, to stress one syllable by placing stress symbol before this syllable, you need to mark all syllables for the word. Or else, the syllable before this stress symbol is stressed. For `sapi`, if you want to stress one syllable, you need to place the stress symbol after this syllable, whether or not all syllables of the word are marked.	Required

phoneme examples

The supported values for attributes of the `phoneme` element were described previously. In the first two examples, the values of `ph="tə.ˈmeɪ.toʊ"` or `ph="təmeɪˈtoʊ"` are specified to stress the syllable `meɪ`.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        <phoneme alphabet="ipa" ph="tə.ˈmeɪ.toʊ"> tomato </phoneme>
    </voice>
</speak>
```

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        <phoneme alphabet="ipa" ph="təmeɪˈtoʊ"> tomato </phoneme>
    </voice>
</speak>
```

XML

```xml
<speak version="1.0" xmlns="https://www.w3.org/2001/10/synthesis" xml:lang="en-
US">
    <voice name="en-US-AvaNeural">
```

```
        <phoneme alphabet="sapi" ph="iy eh n y uw eh s"> en-US </phoneme>
    </voice>
</speak>
```

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        <s>His name is Mike <phoneme alphabet="ups" ph="JH AU"> Zhou </phoneme>
</s>
    </voice>
</speak>
```

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
     <voice name="en-US-AvaNeural">
        <phoneme alphabet='x-sampa' ph='he."lou'>hello</phoneme>
    </voice>
</speak>
```

Custom lexicon

You can define how single entities (such as company, a medical term, or an emoji) are read in SSML by using the phoneme and sub elements. To define how multiple entities are read, create an XML structured custom lexicon file. Then you upload the custom lexicon XML file and reference it with the SSML `lexicon` element.

> **Note**
>
> For a list of locales that support custom lexicon, see footnotes in the language support table.
>
> The `lexicon` element isn't supported by the Long Audio API. For long-form text to speech, use the batch synthesis API (Preview) instead.

Usage of the `lexicon` element's attributes are described in the following table.

Expand table

Attribute	Description	Required or optional
`uri`	The URI of the publicly accessible custom lexicon XML file with either the `.xml` or `.pls` file extension. Using Azure Blob Storage is recommended but not required. For more information about the custom lexicon file, see Pronunciation Lexicon Specification (PLS) Version 1.0.	Required

Custom lexicon examples

The supported values for attributes of the `lexicon` element were described previously.

After you publish your custom lexicon, you can reference it from your SSML. The following SSML example references a custom lexicon that was uploaded to `https://www.example.com/customlexicon.xml`. We support lexicon URLs from Azure Blob Storage. However, note that other public URLs may not be compatible.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
          xmlns:mstts="http://www.w3.org/2001/mstts"
          xml:lang="en-US">
    <voice name="en-US-AvaNeural">
        <lexicon uri="https://www.example.com/customlexicon.xml"/>
        BTW, we will be there probably at 8:00 tomorrow morning.
        Could you help leave a message to Robert Benigni for me?
    </voice>
</speak>
```

Custom lexicon file

To define how multiple entities are read, you can define them in a custom lexicon XML file with either the `.xml` or `.pls` file extension.

> **Note**
>
> The custom lexicon file is a valid XML document, but it can't be used as an SSML document.

Here are some limitations of the custom lexicon file:

- **File size**: The custom lexicon file size is limited to a maximum of 100 KB. If the file size exceeds the 100-KB limit, the synthesis request fails. You can split your lexicon into

multiple lexicons and include them in SSML if the file size exceeds 100 KB.

- **Lexicon cache refresh**: The custom lexicon is cached with the URI as the key on text to speech when it's first loaded. The lexicon with the same URI isn't reloaded within 15 minutes, so the custom lexicon change needs to wait 15 minutes at the most to take effect.

The supported elements and attributes of a custom lexicon XML file are described in the Pronunciation Lexicon Specification (PLS) Version 1.0. Here are some examples of the supported elements and attributes:

- The `lexicon` element contains at least one `lexeme` element. Lexicon contains the necessary `xml:lang` attribute to indicate which locale it should be applied for. One custom lexicon is limited to one locale by design, so if you apply it for a different locale, it doesn't work. The `lexicon` element also has an `alphabet` attribute to indicate the alphabet used in the lexicon. The possible values are `ipa` and `x-microsoft-sapi`.
- Each `lexeme` element contains at least one `grapheme` element and one or more `grapheme`, `alias`, and `phoneme` elements. The `lexeme` element is case sensitive in the custom lexicon. For example, if you only provide a phoneme for the `lexeme` "Hello", it doesn't work for the `lexeme` "hello".
- The `grapheme` element contains text that describes the orthography.
- The `alias` elements are used to indicate the pronunciation of an acronym or an abbreviated term.
- The `phoneme` element provides text that describes how the `lexeme` is pronounced. The syllable boundary is '.' in the IPA alphabet. The `phoneme` element can't contain white space when you use the IPA alphabet.
- When the `alias` and `phoneme` elements are provided with the same `grapheme` element, `alias` has higher priority.

Microsoft provides a validation tool for the custom lexicon that helps you find errors (with detailed error messages) in the custom lexicon file. Using the tool is recommended before you use the custom lexicon XML file in production with the Speech service.

Custom lexicon file examples

The following XML example (not SSML) would be contained in a custom lexicon `.xml` file. When you use this custom lexicon, "BTW" is read as "By the way." "Benigni" is read with the provided IPA "bɛˈniːnji."

```
XML
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<lexicon version="1.0"
      xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://www.w3.org/2005/01/pronunciation-lexicon
        http://www.w3.org/TR/2007/CR-pronunciation-lexicon-20071212/pls.xsd"
      alphabet="ipa" xml:lang="en-US">
    <lexeme>
        <grapheme>BTW</grapheme>
        <alias>By the way</alias>
    </lexeme>
    <lexeme>
        <grapheme>Benigni</grapheme>
        <phoneme>bɛˈniːnji</phoneme>
    </lexeme>
    <lexeme>
        <grapheme>😀</grapheme>
        <alias>test emoji</alias>
    </lexeme>
</lexicon>
```

You can't directly set the pronunciation of a phrase by using the custom lexicon. If you need to set the pronunciation for an acronym or an abbreviated term, first provide an `alias`, and then associate the `phoneme` with that `alias`. For example:

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<lexicon version="1.0"
      xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://www.w3.org/2005/01/pronunciation-lexicon
        http://www.w3.org/TR/2007/CR-pronunciation-lexicon-20071212/pls.xsd"
      alphabet="ipa" xml:lang="en-US">
    <lexeme>
        <grapheme>Scotland MV</grapheme>
        <alias>ScotlandMV</alias>
    </lexeme>
    <lexeme>
        <grapheme>ScotlandMV</grapheme>
        <phoneme>ˈskɒtlənd.ˈmiːdiəm.weɪv</phoneme>
    </lexeme>
</lexicon>
```

You could also directly provide your expected `alias` for the acronym or abbreviated term. For example:

XML

```
  <lexeme>
    <grapheme>Scotland MV</grapheme>
    <alias>Scotland Media Wave</alias>
  </lexeme>
```

The preceding custom lexicon XML file examples use the IPA alphabet, which is also known as the IPA phone set. We suggest that you use the IPA because it's the international standard. For some IPA characters, they're the "precomposed" and "decomposed" version when they're being represented with Unicode. The custom lexicon only supports the decomposed Unicode.

The Speech service defines a phonetic set for these locales: `en-US`, `fr-FR`, `de-DE`, `es-ES`, `ja-JP`, `zh-CN`, `zh-HK`, and `zh-TW`. For more information on the detailed Speech service phonetic alphabet, see the Speech service phonetic sets.

You can use the `x-microsoft-sapi` as the value for the `alphabet` attribute with custom lexicons as demonstrated here:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<lexicon version="1.0"
      xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://www.w3.org/2005/01/pronunciation-lexicon
        http://www.w3.org/TR/2007/CR-pronunciation-lexicon-20071212/pls.xsd"
      alphabet="x-microsoft-sapi" xml:lang="en-US">
  <lexeme>
    <grapheme>BTW</grapheme>
    <alias> By the way </alias>
  </lexeme>
  <lexeme>
    <grapheme> Benigni </grapheme>
    <phoneme> b eh 1 - n iy - n y iy </phoneme>
  </lexeme>
</lexicon>
```

say-as element

The `say-as` element indicates the content type, such as number or date, of the element's text. This element provides guidance to the speech synthesis engine about how to pronounce the text.

Usage of the `say-as` element's attributes are described in the following table.

Expand table

Attribute	Description	Required or optional
`interpret-as`	Indicates the content type of an element's text. For a list of types, see the following table.	Required
`format`	Provides additional information about the precise formatting of the element's text for content types that might have ambiguous formats. SSML defines formats for content types that use them. See the following table.	Optional
`detail`	Indicates the level of detail to be spoken. For example, this attribute might request that the speech synthesis engine pronounce punctuation marks. There are no standard values defined for `detail`.	Optional

The following content types are supported for the `interpret-as` and `format` attributes. Include the `format` attribute only if `format` column isn't empty in this table.

> **Note**
>
> The `characters` and `spell-out` values for the `interpret-as` attribute are supported for all text to speech locales. Other `interpret-as` attribute values are supported for all locales of the following languages: Arabic, Catalan, Chinese, Danish, Dutch, English, French, Finnish, German, Hindi, Italian, Japanese, Korean, Norwegian, Polish, Portuguese, Russian, Spanish, and Swedish.

Expand table

interpret-as	format	Interpretation
`characters`, `spell-out`		The text is spoken as individual letters (spelled out). The speech synthesis engine pronounces: `<say-as interpret-as="characters">test</say-as>` As "T E S T."
`cardinal`, `number`	None	The text is spoken as a cardinal number. The speech synthesis engine pronounces: `There are <say-as interpret-as="cardinal">10</say-as> options` As "There are ten options."

interpret-as	format	Interpretation
`ordinal`	None	The text is spoken as an ordinal number. The speech synthesis engine pronounces: `Select the <say-as interpret-as="ordinal">3rd</say-as> option` As "Select the third option."
`number_digit`	None	The text is spoken as a sequence of individual digits. The speech synthesis engine pronounces: `<say-as interpret-as="number_digit">123456789</say-as>` As "1 2 3 4 5 6 7 8 9."
`fraction`	None	The text is spoken as a fractional number. The speech synthesis engine pronounces: `<say-as interpret-as="fraction">3/8</say-as> of an inch` As "three eighths of an inch."

`date`	dmy, mdy, ymd, ydm, ym, my, md, dm, d, m, y	The text is spoken as a date. The `format` attribute specifies the date's format (*d=day, m=month, and y=year*). The speech synthesis engine pronounces:
		
		`Today is <say-as interpret-as="date">10-12-2016</say-as>`
		
		As "Today is October twelfth two thousand sixteen."
		Pronounces:
		
		`Today is <say-as interpret-as="date" format="dmy">10-12-2016</say-as>`
		
		As "Today is December tenth two thousand sixteen."

`time`	hms12, hms24	The text is spoken as a time. The `format` attribute specifies whether the time is specified by using a 12-hour clock (hms12) or a 24-hour clock (hms24). Use a colon to separate numbers representing hours, minutes, and seconds. Here are some valid time examples: 12:35, 1:14:32, 08:15, and 02:50:45. The speech synthesis engine pronounces: `The train departs at <say-as interpret-as="time" format="hms12">4:00am</say-as>` As "The train departs at four A M."

interpret-as	format	Interpretation

`duration`	hms, hm, ms	The text is spoken as a duration. The `format` attribute specifies the duration's format (*h=hour, m=minute, and s=second*). The speech synthesis engine pronounces:
		
		`<say-as interpret-as="duration">01:18:30</say-as>`
		
		As "one hour eighteen minutes and thirty seconds".
		Pronounces:
		
		`<say-as interpret-as="duration" format="ms">01:18</say-as>`
		
		As "one minute and eighteen seconds".
		This tag is only supported on English and Spanish.

`telephone`	None	The text is spoken as a telephone number. The speech synthesis engine pronounces: `The number is <say-as interpret-as="telephone">(888) 555-1212</say-as>` As "My number is area code eight eight eight five five five one two one two."
`currency`	None	The text is spoken as a currency. The speech synthesis engine pronounces: `<say-as interpret-as="currency">99.9 USD</say-as>` As "ninety-nine US dollars and ninety cents."
`address`	None	The text is spoken as an address. The speech synthesis engine pronounces: `I'm at <say-as interpret-as="address">150th CT NE, Redmond, WA</say-as>` As "I'm at 150th Court Northeast Redmond Washington."
`name`	None	The text is spoken as a person's name. The speech synthesis engine pronounces: `<say-as interpret-as="name">ED</say-as>` As [æd]. In Chinese names, some characters pronounce differently when they appear in a family name. For example, the speech synthesis engine says 仇 in

interpret-as	format	Interpretation
		`<say-as interpret-as="name">仇先生</say-as>` As [qiú] instead of [chóu].

say-as examples

The supported values for attributes of the `say-as` element were described previously.

The speech synthesis engine speaks the following example as "Your first request was for one room on October nineteenth twenty ten with early arrival at twelve thirty five PM."

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        <p>
        Your <say-as interpret-as="ordinal"> 1st </say-as> request was for <say-as
interpret-as="cardinal"> 1 </say-as> room
        on <say-as interpret-as="date" format="mdy"> 10/19/2010 </say-as>, with
early arrival at <say-as interpret-as="time" format="hms12"> 12:35pm </say-as>.
        </p>
    </voice>
</speak>
```

sub element

Use the `sub` element to indicate that the alias attribute's text value should be pronounced instead of the element's enclosed text. In this way, the SSML contains both a spoken and written form.

Usage of the `sub` element's attributes are described in the following table.

Expand table

Attribute	Description	Required or optional
`alias`	The text value that should be pronounced instead of the element's enclosed text.	Required

sub examples

The supported values for attributes of the `sub` element were described previously.

The speech synthesis engine speaks the following example as "World Wide Web Consortium."

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
        <sub alias="World Wide Web Consortium">W3C</sub>
    </voice>
</speak>
```

Mathematical expressions reading

There are two ways to read a mathematical expression:

- With Math domain element,

  Embed the plain text mathematical expression directly in SSML and specify the math domain using `<mstts:prompt domain="Math" />`.

  See the section: Reading plain text mathematical expressions

- With MathML elements

  Represent the mathematical expression with MathML elements.

  See the section: Reading mathematical expressions with MathML

> **Note**
>
> The two features are currently supported in the following locales: de-DE, en-AU, en-GB, en-US, all the sibling locales of English, es-ES, es-MX, all the sibling locales of Spanish, fr-CA, fr-FR, it-IT, ja-JP, ko-KR, pt-BR, and zh-CN.

Reading plain text mathematical expressions

To enable complex mathematical expression reading, you can add `<mstts:prompt domain="Math" />` element to enable math-specific pronunciation rules.

XML

```xml
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
```

```
    <voice name="en-US-AvaMultilingualNeural">
       <mstts:prompt domain="Math" />
       x = (-b ± √(b² - 4ac)) / 2a
    </voice>
</speak>
```

By default, parentheses aren't read out in mathematical expressions. If you'd like the parentheses read out, you can specify `<mstts:mathspeechverbosity level="verbose" />` in SSML

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
    <voice name="en-US-AvaMultilingualNeural">
       <mstts:prompt domain="Math" /><mstts:mathspeechverbosity level="verbose" />
       x = (-b ± √(b² - 4ac)) / 2a
    </voice>
</speak>
```

If you'd like the expression read out in other language with a multilingual voice, specify lang element in SSML.

XML

```
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
   <voice name="en-US-AvaMultilingualNeural">
      <mstts:prompt domain="Math" />
      <lang xml:lang="es-ES">x = (-b ± √(b² - 4ac)) / 2a</lang>
    </voice>
 </speak>
```

Reading mathematical expressions with MathML

The Mathematical Markup Language (MathML) is an XML-compliant markup language that describes mathematical content and structure. The Speech service can use the MathML as input text to properly pronounce mathematical notations in the output audio.

All elements from the MathML 2.0 and MathML 3.0 specifications are supported, except the MathML 3.0 Elementary Math elements.

Take note of these MathML elements and attributes:

- The `xmlns` attribute in `<math xmlns="http://www.w3.org/1998/Math/MathML">` is optional.

- The `semantics`, `annotation`, and `annotation-xml` elements don't output speech, so they're ignored.
- If an element isn't recognized, it'll be ignored, but the child elements within it will still be processed.

The XML syntax doesn't support the MathML entities, so you must use the corresponding unicode characters to represent the entities, for example, the entity `&copy;` should be represented by its unicode characters `&#x00A9;`, otherwise an error occurs.

MathML examples

The text to speech output for this example is "a squared plus b squared equals c squared".

XML

```
<speak version='1.0' xmlns='http://www.w3.org/2001/10/synthesis'
xmlns:mstts='http://www.w3.org/2001/mstts' xml:lang='en-US'>
    <voice name='en-US-JennyNeural'>
        <math xmlns='http://www.w3.org/1998/Math/MathML'>
            <msup>
                <mi>a</mi>
                <mn>2</mn>
            </msup>
            <mo>+</mo>
            <msup>
                <mi>b</mi>
                <mn>2</mn>
            </msup>
            <mo>=</mo>
            <msup>
                <mi>c</mi>
                <mn>2</mn>
            </msup>
        </math>
    </voice>
</speak>
```

Next steps

- SSML overview
- SSML document structure and events
- Language support: Voices, locales, languages

SSML phonetic alphabets

08/07/2025

Phonetic alphabets are used with the Speech Synthesis Markup Language (SSML) to improve the pronunciation of text to speech voices. To learn when and how to use each alphabet, see Use phonemes to improve pronunciation.

Speech service supports the International Phonetic Alphabet (IPA) suprasegmentals that are listed here. You set `ipa` as the `alphabet` in SSML.

Expand table

`ipa`	Symbol	Note
`'`	Primary stress	Don't use single quote (`‘` or `'`) though they look similar.
`,`	Secondary stress	Don't use comma (`,`) though it looks similar.
`.`	Syllable boundary	
`ː`	Long	Don't use colon (`:` or `ː`) though they look similar.
`‿`	Linking	

> **Tip**
>
> You can use the international phonetic alphabet keyboard to create the correct `ipa` suprasegmentals.

For some locales, Speech service defines its own phonetic alphabets, which ordinarily map to the International Phonetic Alphabet (IPA). The eight locales that support the Microsoft Speech API (SAPI, or `sapi`) are en-US, fr-FR, de-DE, es-ES, ja-JP, zh-CN, zh-HK, and zh-TW. For those eight locales, you set `sapi` or `ipa` as the `alphabet` in SSML.

See the sections in this article for the phonemes that are specific to each locale.

> **Note**
>
> The following tables list viseme IDs corresponding to phonemes for different locales. When viseme ID is 0, it indicates silence.

ar-EG/ar-SA

Vowels for ar-EG

Expand table

ipa	viseme	Example 1	Example 2	Example 3
a	2		حرب	شَرَدَ
a:	2		لِسانَهُ	أَدْيانها
i	6		رَجُلٍ	بِهِ
i:	6		كَبيرٌ	أُرْدوازي
u	7		آجُرّ	مُنْذُ
u:	7		آخَرونَ	أَعاقوا

Consonant for ar-EG

Expand table

ipa	viseme	Example 1	Example 2	Example 3
b	21	بازارَ	أَبَ	أَعْطاب
d	19	دابَّة	أَبادَ	أَفْسَد
g	20	جَمَلٌ	رَجَبٌ	مَرَج
k	20	كَآخَر	أَكْلٌ	أَوْراك
t	19	تآخي	مُرَتَّبٌ	أُخْت
dˤ	19	ضاعِن	مُضِرٌّ	مَرَض
q	20	قَمَرٌ	أَتَعاقَد	أَرَق
tˤ	19	طالِبان	أَحاطوهُ	رَبْط
ʔ	19	أُفُقي	فَأْرٌ	السَّماء
f	18	فَنٌّ	يَفِرُّ	شَرَف
h	12	هياجِ	أَحَبَّتْهُ	الْأَبْلَه